

AI Anticipates the Asking Price

City University of Seattle
School of Technology & Computing
TEAM A

Verónica Elze
ElzeVeronica@CityUniversity.edu
Master of Artificial Intelligence

Apoorva Guru
GuruApoorva@CityUniversity.edu
Master of Data Science

Teagan Scharlau
TScharlau@CityU.edu
Doctor of Information Technology

George Stanley
YanHao@CityUniversity.edu
Doctor of Information Technology

Abstract

This project explores the development of a cloud-based environment designed to host artificial intelligence (AI) algorithms for real estate price prediction. Initially, manual local models were developed and evaluated to establish a baseline for prediction accuracy. These models were then compared to those generated by Microsoft Azure's AutoML platform, which provided a more efficient and scalable solution for real-time predictions. The AI models are deployed using containers and served through a Flask-based web interface, offering a user-friendly experience for property value forecasting.

By leveraging Azure's cloud infrastructure, the system ensures seamless scalability and accessibility while managing varying user demand without compromising performance. The AI algorithms analyze historical real estate data and market indicators to generate accurate property price forecasts, offering valuable insights for investors, homebuyers, and real estate professionals.

Throughout the project, a strong emphasis was placed on creating scalable and efficient AI architecture. By starting with manually developed models and transitioning to Azure AutoML, the project demonstrates the advantages of cloud-based deployment in optimizing AI workflows. The integration of Flask and Azure enables streamlined model deployment and improved system reliability.

This project showcases how AI-driven predictive analytics, combined with cloud infrastructure, can enhance decision-making in dynamic markets like real estate, offering a practical and scalable solution for data-driven forecasting.

Keywords: artificial intelligence, machine learning, cloud service, predictive analytics, market predictions, real estate

1. INTRODUCTION

In modern real estate, both investors and potential homeowners face numerous challenges when attempting to predict property values. Market fluctuations and unpredictable trends make it difficult to assess whether property prices are aligned with comparable listings and whether prices are trending up or down. The uncertainty surrounding these factors can lead to missed opportunities for optimal decision-making. To address these time-sensitive challenges, artificial intelligence (AI) offers promising solutions that can enhance real estate decision-making through predictive modeling.

This project aims to develop a cloud-based AI-driven system for real estate price prediction. The system leverages AI algorithms to analyze historical data and market indicators, providing accurate forecasts of property prices. The core objective is to create a scalable environment where AI models can be deployed using Microsoft Azure's AutoML platform and served through a user-friendly web interface built with Flask. The project focuses on combining AI's predictive power with cloud infrastructure to ensure accessibility, scalability, and consistent performance across varying loads.

The key milestones and deliverables of this project include:

- **Development of AI models:** The project began with manually developed local models to establish baseline accuracy, which were later compared to those generated using Azure's AutoML platform for improved performance.
- **Cloud deployment using Microsoft Azure:** The AI models are deployed via Azure containers, ensuring scalability and efficient use of cloud resources.
- **Flask-based web interface:** The system includes a user-friendly interface that allows users to input property details and receive real-time predictions, ensuring ease of use and accessibility.
- **Prediction service:** A reliable prediction service is a core deliverable that provides property value forecasts, helping investors and homebuyers make informed decisions.

By developing an AI-driven, cloud-hosted system, this project demonstrates how predictive analytics can be harnessed to improve decision-making in dynamic markets such as real estate. The system combines the flexibility of AI models with the robustness of cloud infrastructure to deliver a scalable and reliable forecasting solution (Ahmad & Andras, 2019).

This literature review will delve into the background of AI-driven pricing, compare various real estate prediction studies, and examine the

cloud computing systems that support these technologies. By understanding the current landscape and potential hazards associated with AI-driven pricing, stakeholders can make more informed decisions and leverage AI to its fullest potential in the real estate market.

2. LITERATURE REVIEW

The integration of artificial intelligence (AI) in the real estate market has revolutionized how properties are valued and priced. AI-driven pricing leverages sophisticated algorithms to analyze historical transaction data, property features, and market dynamics, generating precise valuations that enhance decision-making processes for real estate professionals (Clement, 2024). This literature review explores AI and predictive analytics in real estate transactions, examining studies and comparing cloud computing systems that support these technologies.

Background on AI-driven pricing

The implementation of AI-driven pricing in the real estate industry presents several potential hazards. This literature review reveals the state of AI and predictive analytics in real estate transactions. Silaghi et al. (2024) conducted a market analysis using deep neural networks trained on Zillow's Zestimate data and other census bureau information to predict price dynamics. They also used a formal parametric model to explain the recursive ripple effects of errors in Market Real Estate Evaluations (MREE) on global inflation.

The study identified several key hazards associated with AI-driven pricing. One significant hazard is the recursive impact of AI products on society and the potential consequences of using AI outputs in training data. This can create a feedback loop where AI-generated price estimations influence market values, affecting future AI predictions and potentially amplifying errors and market distortions.

Understanding the market dynamics influenced by AI price prediction models employed by MREEs like Zillow is crucial. These models can significantly impact market behavior, leading to unintended consequences such as price volatility and market instability. The study also highlights the game-theoretic effects between MREEs and homeowners, particularly in mitigating estimation errors.

In conclusion, while AI-driven pricing offers significant potential for enhancing real estate market efficiency, it also presents several hazards that must be carefully managed. Understanding and addressing these hazards is crucial for leveraging the benefits of AI while minimizing its risks.

Studies about real estate predictions

Predicting rental prices using machine learning and data science has become a significant research area. Two notable studies in this field are "Housing Market Intelligence: Data Science for Rental Price Forecasting" by JS et al. (2024) and "Machine Learning for Rental Price Prediction: Regression Techniques and Random Forest Model" by Raju et al. (2023). Both aimed to enhance rental price prediction accuracy through advanced machine learning models but differed in their approaches and contexts.

According to JS et al. (2024), they applied a Random Forest Regressor algorithm using property attributes for rental price forecasting, discounting other algorithms like CNN, SVM, Logistic Regression, and KNN due to their unsuitability for the specific data characteristics. They collected diverse datasets from real estate platforms, preprocessed the data, and built a model with multiple decision trees, evaluating predictive accuracy using metrics like MAE, MSE, and R2 score.

Raju et al. (2023) focused on machine learning techniques, regression analysis, and a Random Forest model to predict house rental prices. They used Multiple Linear Regression to model relationships between variables, Ridge Regression to prevent overfitting, and LASSO Regression for feature selection and model simplification. Additionally, they leveraged other regression models like Elastic Net, Gradient Boosting, and Ada Boost Regression for regularization and minimizing errors.

Both studies emphasize the importance of accurate, data-driven methods for forecasting rental prices, which use historical rental data and various features to make predictions. However, they differ in their specific techniques and publication contexts. Raju et al. (2023) explored a variety of regression techniques, while JS et al. (2024) focused, specifically on the Random Forest Regressor. Despite these differences, both contribute valuable insights into the application of machine learning for rental price prediction.

Cloud Computing Comparisons

Amazon Web Services (AWS) and Microsoft Azure are leading cloud platforms, essential for scalability, efficiency, and innovation. While both provide a wide range of services, each has its own distinct advantages.

Mare et al. (2023) highlighted AWS's vast storage options and pay-as-you-go pricing, while Azure provides competitive storage with discounts for Microsoft customers. Singh (2024) discussed AWS Code Pipeline's integration with Azure DevOps using OpenID Connect, automating build, test, and deploy phases, and linking with GitHub

for automatic triggers. Biradar and Moolya (2022) investigated how GitHub Actions integrate with AWS for CI/CD pipelines, using AWS CodeBuild and AWS CodeDeploy for automated builds and deployments.

Both platforms offer computing power, storage, and databases for managing applications at scale. AWS provides Amazon EC2 for scalable virtual servers and AWS Lambda for serverless computing, while Azure offers Virtual Machines and Azure Functions. Both support auto-scaling to adjust resources based on demand.

Integration with DevOps tools is seamless, facilitating continuous integration and deployment. AWS and Azure prioritize security and compliance, offering encryption, identity and access management, and compliance certifications. AWS's early market entry and extensive service array give it a larger global infrastructure footprint and a mature ecosystem. Azure's strong integration with Microsoft's ecosystem appeals to enterprises using Microsoft products.

AWS offers managed services like AWS CodeBuild and AWS CodeDeploy, automating build and deployment processes and integrating with GitHub Actions. Azure offers similar services with Azure DevOps, supporting CI/CD pipelines and integrating with GitHub. AWS's offerings include specialized services for machine learning, analytics, and IoT, while Azure offers competitive alternatives like Azure Machine Learning, Azure Synapse Analytics, and Azure IoT Hub.

Both AWS and Azure provide powerful cloud computing platforms with extensive services, scalability, and robust security. Their differences in service integration, offerings, and user experience highlight their unique strengths. Businesses must consider these factors when choosing the right cloud service provider to meet their needs and objectives.

In summary, this literature review delved into the background of AI-driven pricing, compared various real estate prediction studies, and examined the cloud computing systems that underpin these technologies. By understanding the current landscape and potential hazards associated with AI-driven pricing, stakeholders can make informed decisions and leverage AI to its fullest potential in the real estate market.

Connection to the Project

The findings from these studies directly influenced key decisions in our project. Based on the demonstrated success of AI algorithms in real estate prediction, we incorporated machine learning models, initially developing them locally before transitioning to Microsoft Azure's AutoML platform for greater scalability and ease of

deployment. Additionally, the comparison of cloud platforms in the literature informed our choice of Azure over AWS due to Azure's integrated AutoML services, which provided a more streamlined approach for model training and deployment within our project's timeline. These decisions reflect how the literature shaped both the technical strategies and the infrastructure chosen for our real estate prediction system.

3. METHODOLOGY

Methods Applied

This project began with an extensive literature review, exploring current AI methodologies in real estate and cost predictions. Using these insights, we developed an AI model capable of predicting property values. We designed an interface to input property details and receive predictions. The performance of the model was tested for both accuracy and prediction time, with comparisons made against a machine learning model created using Microsoft Azure's AutoML tools.

Following this, data analysis was conducted, and the findings were formalized, as presented in this report.

Tools & Technologies

The dataset for training and testing the AI model was sourced from Kaggle, a widely recognized platform for housing market research and data analytics (Brar, 2024). The AI model was built using the Python programming language, incorporating libraries such as TensorFlow, Scikit-learn, Flask, NumPy, and Pandas. Development took place in both Microsoft Visual Studio Code (VS Code) and Google Colaboratory (Colab).

We employed a regression-based algorithm, optimized for predicting continuous numerical values based on user input. The interface was built using Flask and HTML, with JSON managing data transmission. The system was designed to be compatible with modern web browsers, including Firefox, Chrome, and Edge.

The user interface (UI) and backend were constructed using Flask, with data submitted through curl and JSON for efficiency and ease of development.

System Architecture and User Workflow

The real estate prediction system combines a user-friendly interface with a robust backend architecture that integrates Microsoft Azure's AutoML for cloud-based machine learning predictions. The system consists of HTML forms for user input, Flask as the backend framework, and Azure for generating property value predictions.

- **User Interface** (index.html): The user interface begins with a form where users input property details, including bathrooms, square footage, and condition ratings. Advanced inputs such as latitude, longitude, and neighborhood data are also collected to ensure personalized predictions. The interface is designed for ease of use and features validation to ensure accuracy. It is accessible across multiple devices, ensuring users can interact with the system without technical challenges. Upon form submission, the data is sent to the backend for processing. The responsive design ensures usability across various browsers and devices.

- **Backend Processing** (app.py): The Flask-based backend handles the logic and sends user inputs to a machine learning model hosted on Microsoft Azure. The inputs are structured in a JSON format, and Flask handles the secure communication between the user interface and the machine learning model through a REST API endpoint (GitHub Docs, 2024). This backend leverages Azure's cloud infrastructure to generate predictions in real-time based on the inputs received. The data is securely transmitted to the Azure machine learning service, which processes the input using a pre-trained model, providing a predicted property value. This model interaction is authenticated using API keys, ensuring secure communication (Mansour et al., 2023).

- **Displaying Results** (result.html): Once the prediction is returned, the result is presented on a simple results page that displays the predicted property value in an easy-to-read format. A "Go Back" button is available for users to return to the form and submit new queries. This workflow ensures that users experience seamless interaction with the AI-powered property valuation tool. The integration with Azure's scalable platform allows the system to handle real-time predictions efficiently, providing valuable insights to users in the real estate market.

Cyber Security Implementation

In the development and deployment of the AI models for market analysis and price prediction, some cybersecurity measures were put in place to keep the system safe (Zhang et al., 2010):

- **API Security:** The Flask-based API is secured using key-based authentication to control access and prevent unauthorized use. In addition, to ensure data integrity and prevent malicious inputs, the web form incorporates input validation techniques that sanitize and verify user entries before they are processed by the API, reducing the risk of injection attacks or invalid data submissions. These combined measures help safeguard the system from common web vulnerabilities.

- **Authentication and Authorization:** A key-based authentication mechanism was implemented for the deployed AI model in Microsoft Azure. This ensures that only users with a valid API key can make requests to the AI model, providing a level of access control to the system's resources.
- **Cloud Platform Security:** The Microsoft Azure platform, which hosts the AI models, provides built-in security features, such as encrypted storage and secure connections, though specific encryption strategies for the data are not explicitly mentioned in the paper.
- **Deployment Security:** During the deployment of the AI model, basic security practices were followed, including ensuring secure communication between the user interface, the Flask API, and the Azure-hosted model. This minimizes unauthorized access and protects the integrity of data communications.

4. AI MODEL DEVELOPMENT PROCESS

Our AI model development began with the manual creation of machine learning models to establish a solid foundation. This hands-on approach allowed us to carefully analyze the dataset and tailor the models to our project's needs. After evaluating these manually developed models, we transitioned to using Microsoft Azure's AutoML platform. This automated method streamlined model selection and hyperparameter tuning, improving both efficiency and overall performance.

Manual Model Training Process

We manually developed our machine learning models through a systematic approach, starting with the creation of linear regression, random forest, and K-Nearest Neighbors (KNN) models using Python. The following steps detail the process:

Step 1 – Exploration & Data Analysis

The purpose of this step was to examine the dataset, understand its structure and distribution, as well as locate any potential issues.

In machine learning and data analytics, this step is crucial for identifying patterns, locating outliers, and detecting data quality issues during preprocessing.

The initial data load and view of the first five records of the dataset plus the functions executed to examine the data can be found in the appendix as Figures 1, 2, and 3.

We then looked at the distribution of the data set using our target variable of Price:

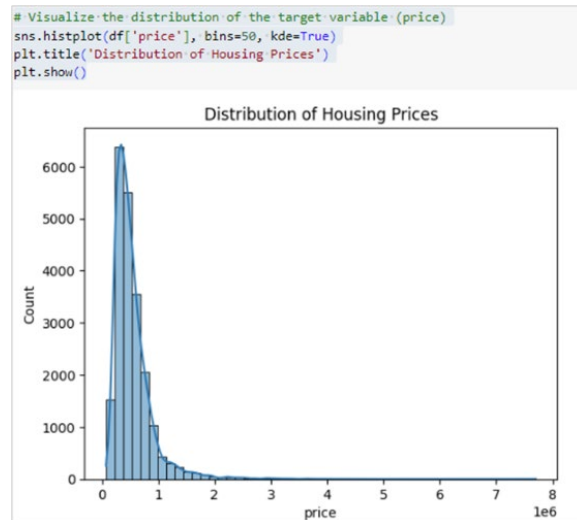


Figure 4 – Distribution of house prices

The correlation of variables within the dataset was the final metric we examined. This matrix can be found in the appendix as Figure 5 Visualization of the Correlation Matrix.

Step 2 – Data Preparation

During this step, the team cleaned and performed transformations on the data to prepare it for data modeling. This ensured that the dataset is suitable for our ML algorithm, addresses missing values and encodes categorical variables, while normalizing numerical features. Figure 6 shows the step where we ran the correlation matrix to help identify the most significant features in the dataset.

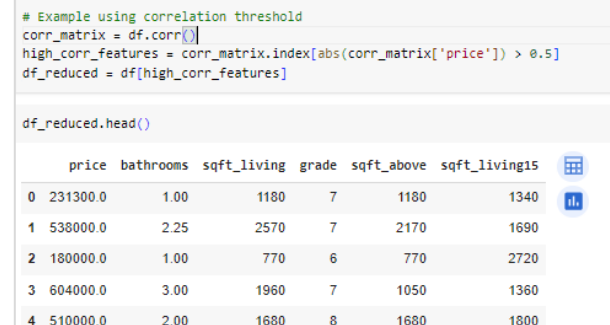


Figure 6 – Feature reduction based on correlation matrix

Step 3 – Model Creation & Training

This step was the core of our project process, in which we created the AI/ML model and trained it on a subset of the data to prepare it for use in the final product. For this project we created different models to test the effectiveness of the models. These models were a linear regression model, random forest model, and KNN model.

```
# Initialize models
linear_model = LinearRegression()
random_forest_model = RandomForestRegressor(n_estimators=100, random_state=42)
knn_model = KNeighborsRegressor(n_neighbors=5)

# Separate features and target variable
X = df_reduced.drop('price', axis=1)
y = df_reduced['price']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the models
linear_model.fit(X_train, y_train)
random_forest_model.fit(X_train, y_train)
knn_model.fit(X_train, y_train)
```

Figure 7 - Creating and training the models.

As shown, we split 20% of the dataset into a testing set to run the model against during post-training testing to determine accuracy. We then concluded this step by training the models.

Step 4 – Testing

Machine learning models are not of any use if they are inaccurate or produce unexpected results, and as such one of the most important steps in model development was testing. As a result, we tested the models using a test set that had been generated from the overall dataset.

```
# Predictions
linear_preds = linear_model.predict(X_test)
rf_preds = random_forest_model.predict(X_test)
knn_preds = knn_model.predict(X_test)

# Evaluation
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_preds))
linear_r2 = r2_score(y_test, linear_preds)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_preds))
rf_r2 = r2_score(y_test, rf_preds)

knn_rmse = np.sqrt(mean_squared_error(y_test, knn_preds))
knn_r2 = r2_score(y_test, knn_preds)

print(f'Linear Regression RMSE: {linear_rmse}, R^2: {linear_r2}')
print(f'Random Forest RMSE: {rf_rmse}, R^2: {rf_r2}')
print(f'KNN RMSE: {knn_rmse}, R^2: {knn_r2}')
```

Figure 8 – Predicting the models on test data.

This produced the results as shown in Figure 9.

```
Linear Regression RMSE: 261392.1693005368, R^2: 0.5480397400391172
Random Forest RMSE: 262715.474127974, R^2: 0.5434520349336687
KNN RMSE: 273653.16468823556, R^2: 0.5046455672573993
```

Figure 9 – RMSE and R² values of each model

The RMSE and R² values show model performance. As we can see the R² shows that the model is about a 50-55% fit to the data, and that in conjunction with the RMSEs means we can do some more to improve the models.

Step 5 – Compare the Performance of the Different Models

To compare our various models, we plotted the predicted values against the actual values. The results for each model are displayed in the following three figures:

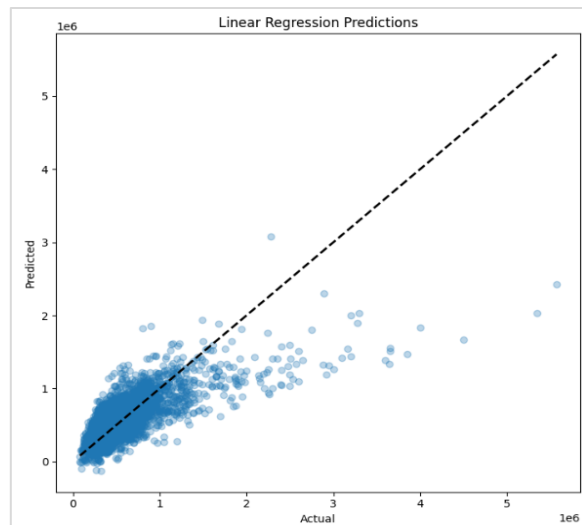


Figure 10 – Prediction Plot: Linear Regression Model

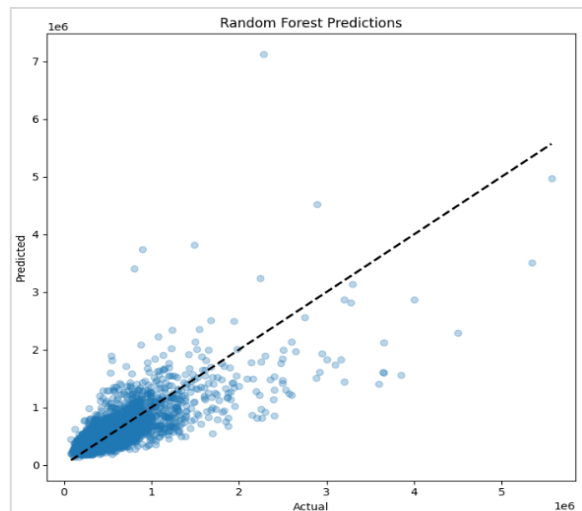


Figure 11 – Prediction Plot: Random Forest

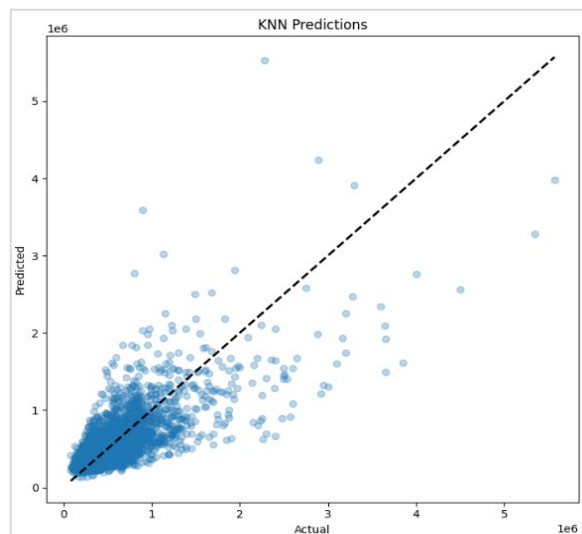


Figure 12 – Prediction Plot: KNN

As we can see from the above plots, the KNN model seems to fit the data better than the other two models and the data tends to flow along the trend line in the chart, while the other two have points that are away from the line of best fit.

Considering performance metrics like RMSE (Root Mean Square Error) and R^2 (R-Squared), the linear regression model demonstrated superior performance compared to the others.

Microsoft AutoML model:

Microsoft's AutoML streamlines the entire process of implementing machine learning solutions for practical applications. It simplifies model development by automatically selecting algorithms, tuning hyperparameters, and generating models without requiring deep expertise in machine learning. Integrated with Azure's AI tools, AutoML is both accessible and scalable for small teams up to enterprise scale applications.

The team selected Azure's AutoML over AWS because it accelerates the model-building process, reduces human error, and allows developers and data scientists to focus on higher-level tasks. This increases productivity and ensures more consistent, accurate results. The team believes that leveraging this cloud platform allowed us to achieve the best-performing model in the limited time available for this project.

We followed the same steps as we did with our manually developed model. All figures in this section are available larger in *Appendix B – Microsoft Azure AutoML Process – Figures and Images*.

Step 1 – Create a new workspace

The first step in creating a new AutoML process in Azure involves creating a new workspace in the Azure portal. This houses all the experiments, datasets, jobs, compute assets, as well as others.

Step 2 – Create an Experiment Job

Creating the ML model in Azure is simply a matter of running a wizard and following the steps described in this section of the document. A walkthrough of the steps completed by the development team is documented here. Figure 13 shows the options for what kind of training you want to choose for your model.

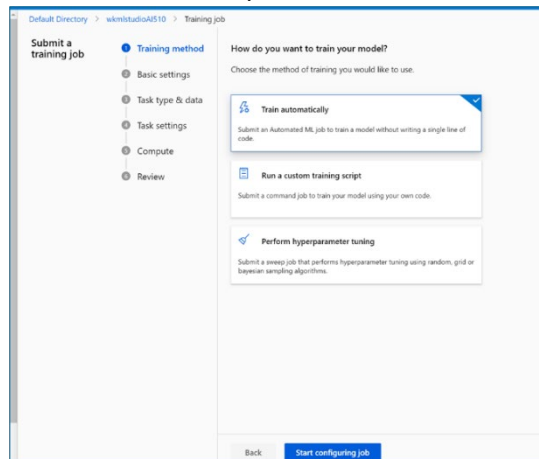


Figure 13 – Create training wizard dialog- step 2

The figure shows our task choice to use the Train Automatically option to speed up development time.

Step 3 – Basic Information and task type

The next steps in the process were to give basic information to the system and select the task type to perform. Because this is a prediction, we chose a regression task type as seen in Figure 14.

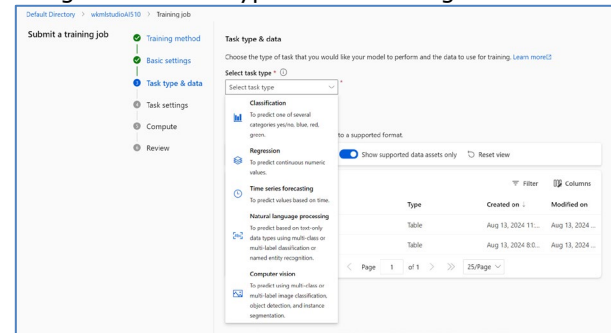


Figure 14 - Create training wizard dialog – step 3

Step 4 – Create the Dataset

The next step in the process was to create the dataset. In this case we used the same dataset from Kaggle as for our other ML model. The data was a tabular type as seen in Figure 15.

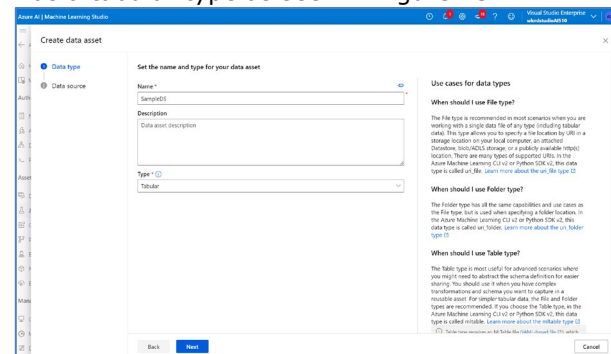


Figure 15 – Create dataset – data type

As part of this step, we also selected the data source, which was the CSV file acquired from Kaggle. We then specified settings as seen in Figure 16. This included the file format, delimiter of the data, type of encoding, information about headers, and whether we should skip rows. We also got to see a preview of the data as it was at that point. This allowed us to verify that the schema for the data was properly imported.

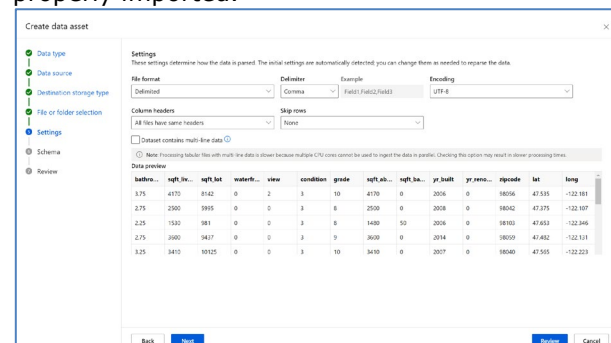


Figure 16 – Data selections, settings & schema

Step 5 – Choose Target Field

Next, we chose the target column, which represents the variable we aimed to predict with our regression model. In this instance, the target was the house price. The price of a house is influenced by factors like the number of bathrooms, square footage, and other metrics. This is illustrated in Figure 17.

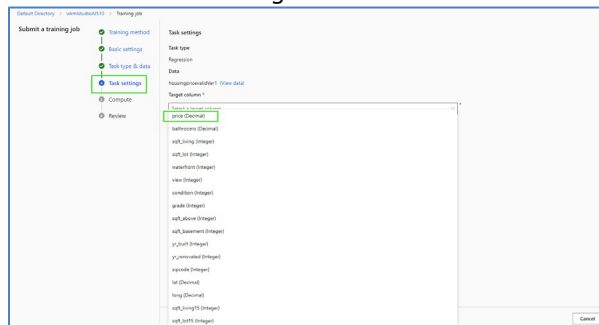


Figure 17 – Select target for predictions.

Step 6 – Additional Settings Selection

The next setup step involves selecting additional configuration settings. In this step, we chose to have it explain the best model and use all supported models. We could have limited which types of models to use, but we decided to not limit our options. See Figure 18 view of this screen.

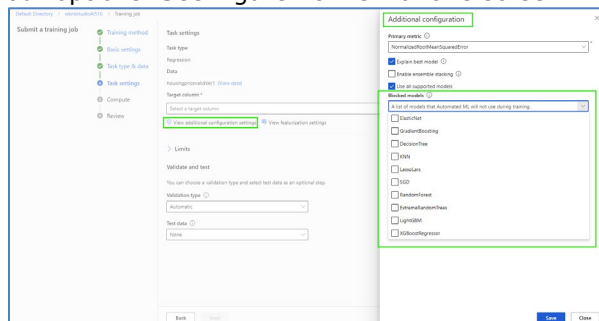


Figure 18 – Select additional settings.

Step 7 – Validation and Test-Train Split

The last step before training the model was the test train split and validation selection. In this step, we chose to use an automatic validation type rather than specifically selecting one of the assorted options. The team decided to let the system choose the best option. See Figure 19.

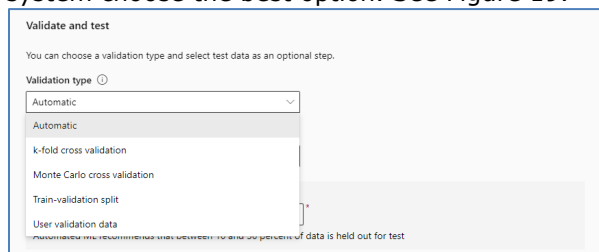


Figure 19 – Validation type selection

Next, we selected the test train data split. AutoML recommended that we use between 10 and 30% of the data as the test, so we chose to use our class norm of 20%. Figure 20 demonstrates what this dialogue looks like and the values used.

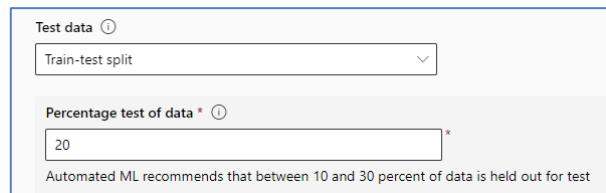


Figure 20 – Test-train split settings

Step 8 – Compute Selection

Microsoft Azure has multiple options for compute resources for an AI project. The choices were a compute cluster, compute instance, or serverless option. We chose a compute instance for the computational needs of the project, which is shown in Figure 21.

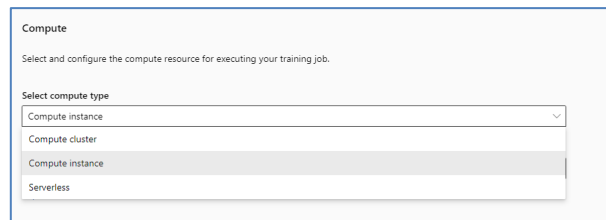


Figure 21 – Compute option selection

Step 9 – Job Submission

Finally, after all the configuration was complete, we were able to submit the job to run in the cloud. Figure 22 illustrates that it took approximately 31 minutes for the job to run.

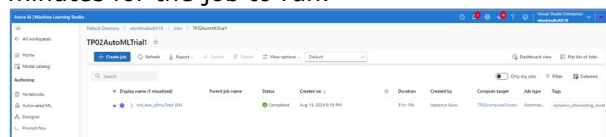


Figure 22 – Job Submitted

Figure 23 shows the job running in the dashboard overview.

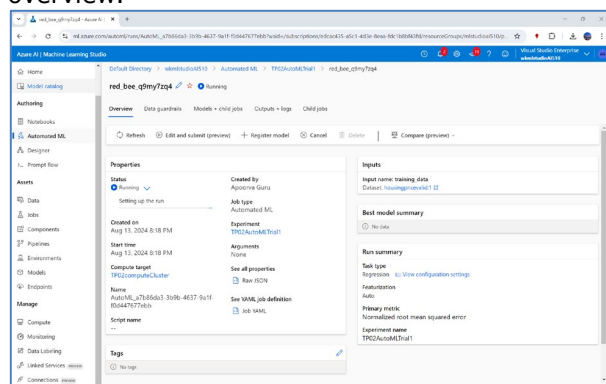


Figure 23 – Overview of running job

You can see data about the job in this overview. Some examples are status, creator, job type, creation and start times, arguments (if any), and compute target. This overview also shows what the training data inputs and a summary of the run. Since this was the first run, and the job was still running, it was not the best model summary overall.

Step 10 – Monitor Job & Model Selection

At this point, AutoML was trying multiple different regression models. Figure 24 and 25 show the list of potential models as well as their normalized root mean squared error values.

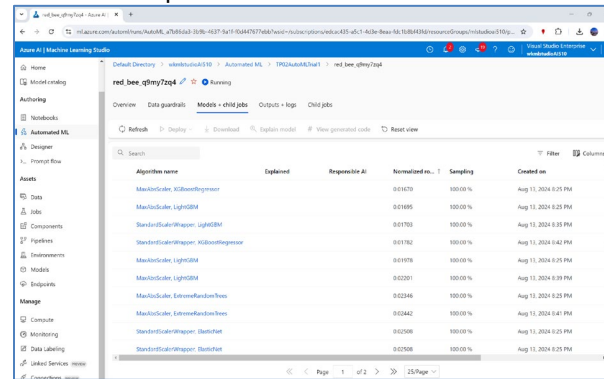


Figure 24 – AutoML model choices – pg. 1 of 2

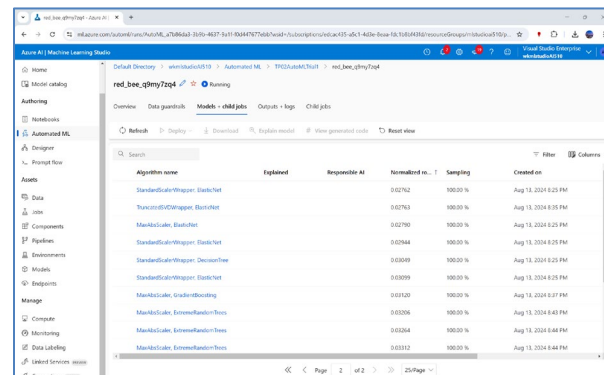


Figure 25 – AutoML model choices – pg. 2 of 2

At the end of the job, the *VotingEnsemble* model had the lowest normalized root mean squared error (RMSE) value out of all the tested models. This means its predictions are closest to the actual values, implying the highest accuracy. See Figure 26.

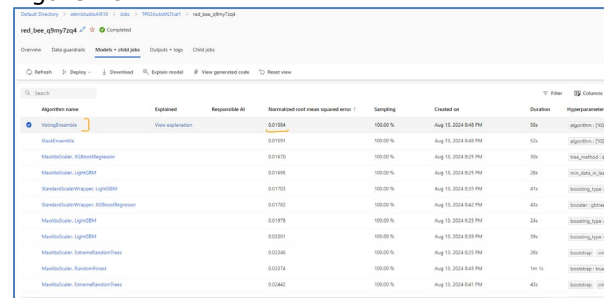


Figure 26 – Best model based on normalized Root Mean Squared Error value.

In this task, we also evaluated the metrics shown in Figure 27, such as the Normalized Median Absolute Error, which indicates the median of the absolute differences between predicted and actual values.

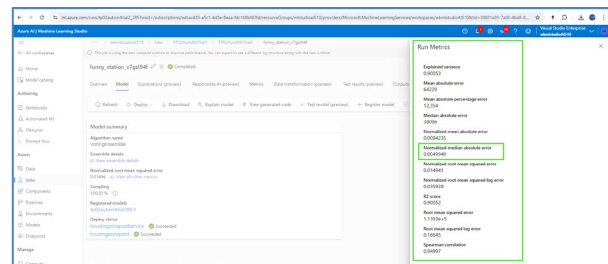


Figure 27 – Additional metrics

As part of our additional examination of the model, we looked at the top 10 features, sorted by their importance. Figure 28 identifies Latitude and Square Footage of Living space as having the most noteworthy influence on the model's prediction.

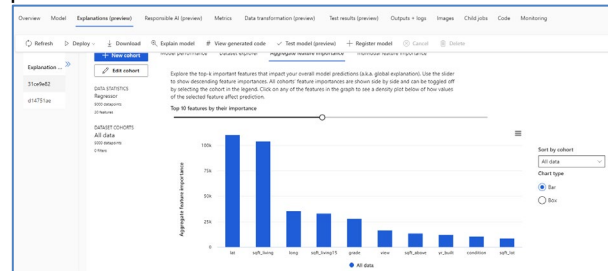


Figure 28 – Top 10 features sorted by relevance (highest to lowest)

Step 11 – Examine Feature Importance

Our next step was to consider how the most important feature, Latitude, is highly correlated to predicted Price. The generated scatter plot is shown in Figure 29.

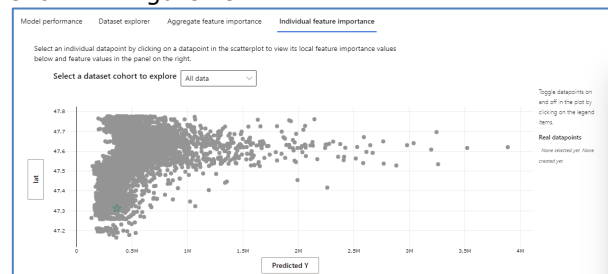


Figure 29 – Latitude vs Price: Most significant feature

The lack of correlation between Year Renovated and Price is a stark contrast from the Latitude correlation. This makes sense since it was the least significant feature. Azure generated Figure 30, which is a scatter plot graph with Year Renovated and Price.

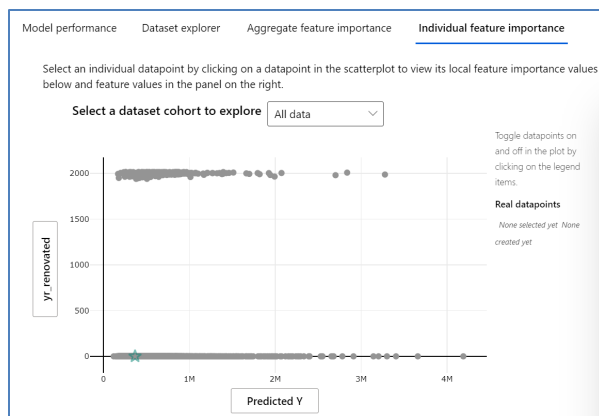


Figure 30 – Year-Renovated vs Price: Least significant feature

Step 12 – Data Revision

Based on the feature importance metric the team decided to remove the bottom four features, which created a new data set. The steps above were repeated on the new data set with a new job. Figure 31 shows which features were removed from the data set. Bedrooms and floors were an unexpected low relevance and attributed to a bias in this particular data set.

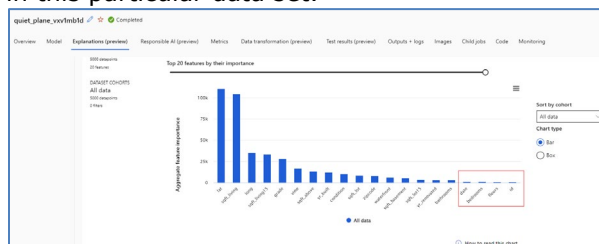


Figure 31 – Features removed based on top 20 features by importance.

Step 13 – Model Deployment

The next step was the deployment of the model. Microsoft Azure allows for a simple form-based deployment process. For security purposes we chose to enable a key based authentication system. Figure 32 shows the deployment settings.

Figure 32 – Model deployment & access security settings

Figure 33 shows the successfully deployed model available for predictions.

Model name	Version	Deployment status	Deployment date	Deployment time	Deployment size	Deployment location
tp02automltrial2851	1	Deployed	Aug 14, 2024 12:00 PM	10m	100MB	tp02automltrial2851

Figure 33 – Successful and secure deployment of model

ML Endpoint and Authentication

The deployed service can be accessed programmatically via HTTP requests, secured through key-based authentication. This mechanism ensures that only users with a valid API key can interact with the model, safeguarding the service from unauthorized access.

As illustrated in Figure 34, remember, the interface for this model is a web-based form, built using HTML and CSS, with a Flask-based API handling the communication between the front end and the model.

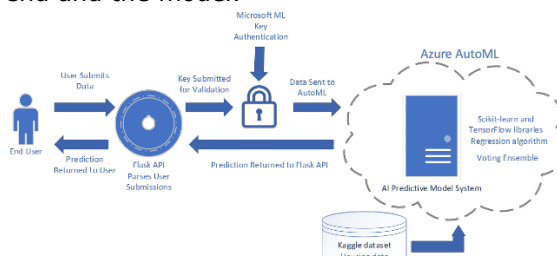


Figure 34 – Key-Based Authentication process

In Figure 35, we see the final deployment details, showing the model running on an Azure virtual machine and accessible through the REST API endpoint.

Model name	Version	Deployment status	Deployment date	Deployment time	Deployment size	Deployment location
housingpricepredservice	1	Deployed	Aug 14, 2024 12:00 PM	10m	100MB	housingpricepredservice

Figure 35 – AutoML web service details including REST endpoint.

5. FINDINGS

During our investigations, the team found the following items to be of importance to the question at hand:

Effectiveness of AI Algorithms

The project explored various AI algorithms for market analysis, price prediction, and property

valuation. Linear Regression and Random Forest models provided solid baselines. However, the models developed using Microsoft Azure's AutoML platform demonstrated superior predictive accuracy. AutoML streamlined the model selection process, enabling faster iterations and more refined outcomes.

Impact on Market Insights and Predictions

The AI models generated insightful market predictions, particularly in identifying property values and trends. These insights empower investors and homebuyers to make informed decisions. The accuracy of the models in predicting market prices and trends illustrates the significant role that AI can play in enhancing real estate decision-making.

Comparison of AutoML with Manual Model Development

A comparison between manually developed models and those generated via AutoML revealed that the latter matched—and often exceeded—the performance of manually tuned models. This was particularly evident in reduced prediction errors and improved robustness across diverse datasets. AutoML's efficiency also allowed the team to focus more on strategic decisions rather than the intricacies of model tuning.

Scalability and Performance

Implementing the AI models on cloud platforms, particularly through Microsoft Azure, demonstrated the system's ability to scale effectively. The cloud infrastructure handled large datasets and numerous concurrent users without significant performance degradation, which is crucial for real-world applications where demand can fluctuate.

Usability of the Flask-Based Interface

The Flask-based web interface allows users to input property details, such as bathrooms, square footage, and other relevant features, through a simple and intuitive form. After submission, the interface processes the input data and dynamically displays the predicted property value on a dedicated results page. The design ensures users can easily navigate back to enter details and receive additional predictions. With its responsive layout, the interface remains accessible across various devices and browsers, providing a seamless and efficient user experience.

Challenges and Limitations

While the project achieved its objectives, challenges were encountered, particularly with data quality, such as missing values and outliers, which required extensive preprocessing. Additionally, although AutoML reduced the model selection burden, domain-specific knowledge was still necessary to interpret the results effectively.

6. FUTURE WORK

Several improvements can enhance the AI-driven real estate prediction system developed in this project. Improving model accuracy by exploring advanced algorithms like neural networks or Gradient Boosting Machines, along with hyperparameter tuning, would yield more accurate predictions. Incorporating real-time data, such as live listings and economic indicators, as well as geospatial and environmental data, would refine the system's analysis capabilities.

On the security front, defenses against adversarial attacks should be prioritized. Implementing adversarial training and input validation will help safeguard model integrity. Enhancing model transparency using tools like SHAP or LIME will ensure better explainability and secure decision-making in high-stakes applications.

User experience improvements can be achieved by creating more interactive and personalized interfaces, enabling dynamic input adjustments and customized insights. Ethical considerations such as bias detection and mitigation should also be explored to ensure fair property valuations.

In terms of scalability, adopting container orchestration tools like Kubernetes would optimize AI deployments for large-scale use, while edge computing could improve real-time processing performance. Strengthening privacy and data security with encryption methods and implementing differential privacy techniques would further protect sensitive user and market data. Additionally, more granular access controls and detailed logging mechanisms would enhance system security.

By pursuing these future ideas, the AI system can become more accurate, scalable, secure, and applicable to a broader range of markets and use cases.

7. CONCLUSION

This project successfully demonstrated the integration of AI-driven models into a cloud-based environment for real estate price prediction. By leveraging Microsoft Azure's AutoML platform, we achieved efficient model training and deployment, reducing manual intervention while maintaining high predictive accuracy. The system's deployment via containers on Azure ensures scalability, allowing it to manage varying user demands without performance degradation.

The Flask-based web interface provided a user-friendly method for accessing the AI models, enabling real-time predictions for property values. This design allows users, including investors and homebuyers, to easily interact with the system and make informed decisions based on dependable, data-driven forecasts.

Through careful management of cloud resources, the project balanced cost-effectiveness with scalability. Azure's infrastructure allowed the system to deliver consistent performance, while the use of pre-trained models further optimized resource usage. Additionally, robust security measures, such as key-based authentication and input validation, were implemented to protect the system and user data.

In summary, this project achieved its core objectives by creating a practical, scalable, and accessible AI system that demonstrates the potential of cloud-based predictive analytics in the real estate market.

8. REFERENCES

- Ahmad, A. A., & Andras, P. (2019). Scalability analysis comparisons of cloud-based software services. *Journal of Cloud Computing Advances Systems and Applications*, 8(1). <https://doi.org/10.1186/s13677-019-0134-y>
- Biradar, M., & Moolya, S. (2022, March 29). *Integrating with GitHub Actions - CI/CD pipeline to deploy a Web App to Amazon EC2*. Amazon Web Services. <https://aws.amazon.com/blogs/devops/integrating-with-github-actions-ci-cd-pipeline-to-deploy-a-web-app-to-amazon-ec2/>
- Brar, S. S. (2024). *Housing Price Dataset*. Kaggle.com. <https://www.kaggle.com/datasets/sukhmandeep Singhbrar/housing-price-dataset/code>
- Clement, G. (2024, April 15). AI in Real Estate: Trends & Use cases. *Realeflow*. <https://blog.realeflow.com/artificial-intelligence-real-estate>
- GitHub Docs. (2024). *Deploying Java to Azure App Service - GitHub Docs*. GitHub Docs. <https://docs.github.com/en/actions/deployment/deploying-to-your-cloud-provider/deploying-to-azure/deploying-java-to-azure-app-service>
- Brar, S. S. (2024, April 4). *Housing Price Dataset*. Kaggle. <https://www.kaggle.com/datasets/sukhmandeep Singhbrar/housing-price-dataset>
- JS, N., Sravya, S. G., & Tharun, K. (2024, March 1). *Housing Market Intelligence: Data Science for rental Price Forecasting*. IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/abstract/document/10511646>
- Mansour, F. A. M., Mare, F. A. M., & Abughalyah, A. M. (2023). Amazon AWS vs. Microsoft Azure: Two Prominent Cloud Service Platforms. *International Science and Technology Journal*, 33rd (1). ResearchGate. https://www.researchgate.net/publication/37774113_Amazon_AWS_vs_Microsoft_Azure_Two_Prominent_Cloud_Service_Platforms
- Nirmala JS, Somu Geetha Sravya, & Kamsala Tharun. (2024). *Housing Market Intelligence: Data Science for Rental Price Forecasting. 2024 3rd International Conference for Innovation in Technology (INOCON), 2024*. <https://doi.org/10.1109/inocon60754.2024.10511646>
- OpenAI. (2024, September 7). Response to the question, "How can I improve my writing with a collegiate tone and the least possible changes to my original text?" ChatGPT. <https://chat.openai.com/>
- Raju, R., Arham Neyaz, Ahmed, A., & Singh, A. (2023). Machine Learning for Rental Price Prediction: Regression Techniques and Random Forest Model. *Social Science Research Network*. <https://doi.org/10.2139/ssrn.4587725>
- Silaghi, V., Alssadi, Z., Mathew, B., Alotaibi, M., Alqarni, A., & Silaghi, M. (2024). Modeling the Feedback of AI Price Estimations on Actual Market Values. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2405.18434>
- Singh, R. (2024, July 29). *Use OpenID Connect with AWS Toolkit for Azure DevOps to perform AWS CodeDeploy deployments | Amazon Web Services*. Amazon Web Services. <https://aws.amazon.com/blogs/devops/use-openid-connect-with-aws-toolkit-for-azure-devops-to-perform-aws-codedeploy-deployments/>
- Zhang, Q., Cheng, L., & Boutaba, R. (2010). *Cloud computing: State-of-the-art and research challenges*. *Journal of Internet Services and Applications*, 1(1), 7–18. <https://doi.org/10.1007/s13174-010-0007-6>
- Zillow Group. (2024, January 17). *Our AI principles*. Zillowgroup.com. <https://www.zillowgroup.com/our-ai-principles/>

Appendix A

Manual Model Figures and Images

```
# Load the dataset
url = "Housing.csv"
df = pd.read_csv(url)
df = df.drop(['id', 'date'], axis=1)

[54] df.head()
```

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15	sqft_lot15
0	231300.0	2	1.00	1180	5650	1.0	0	0	3	7	1180	0	1955	0	98178	47.5112	-122.257	1340	5650
1	538000.0	3	2.25	2570	7242	2.0	0	0	3	7	2170	400	1951	1991	98125	47.7210	-122.319	1690	7639
2	180000.0	2	1.00	770	10000	1.0	0	0	3	6	770	0	1933	0	98028	47.7379	-122.233	2720	8062
3	604000.0	4	3.00	1960	5000	1.0	0	0	5	7	1050	910	1965	0	98136	47.5208	-122.393	1360	5000
4	510000.0	3	2.00	1680	8080	1.0	0	0	3	8	1680	0	1987	0	98074	47.6168	-122.045	1800	7503

Figure 1 - Data Load and Display

```
# Display basic information
print(df.info())
print(df.describe())
```

Figure 2 - Display metadata of loaded data.

```
# Check for missing values
print(df.isnull().sum())
```

Figure 3 - Check for nulls of laded dataset.

```
# Visualize the distribution of the target variable (price)
sns.histplot(df['price'], bins=50, kde=True)
plt.title('Distribution of Housing Prices')
plt.show()
```

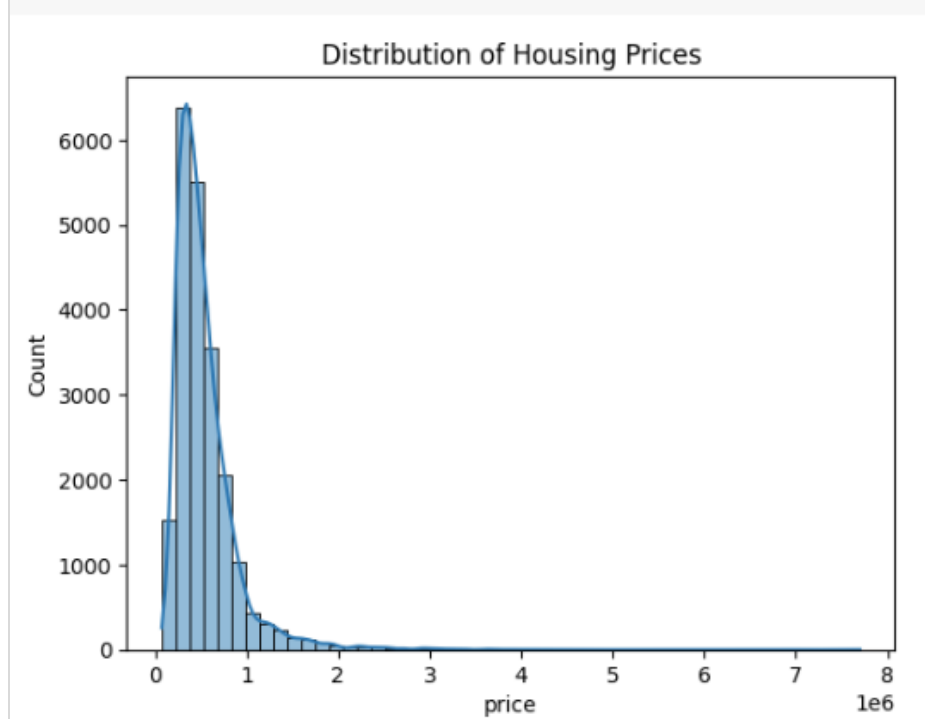


Figure 4 Distribution of house prices.

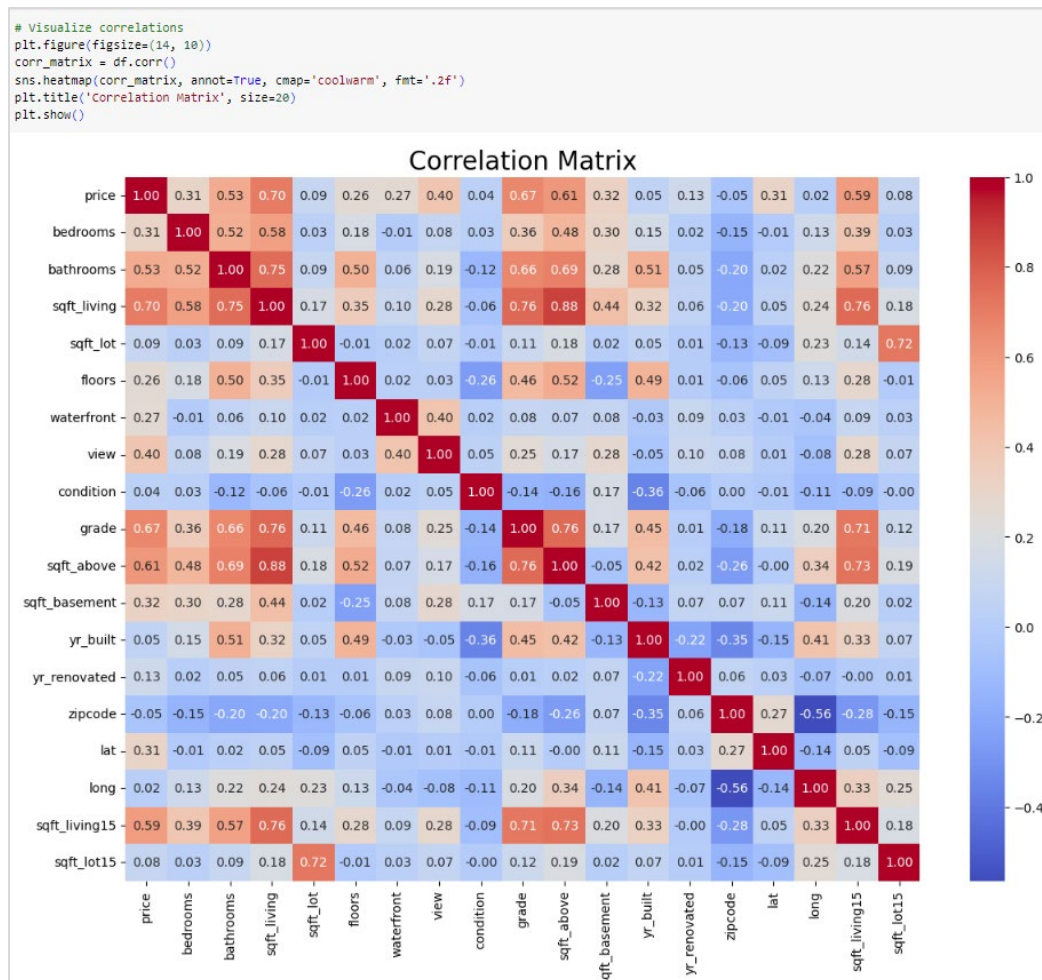


Figure 5 - Visualization of the correlation matrix.

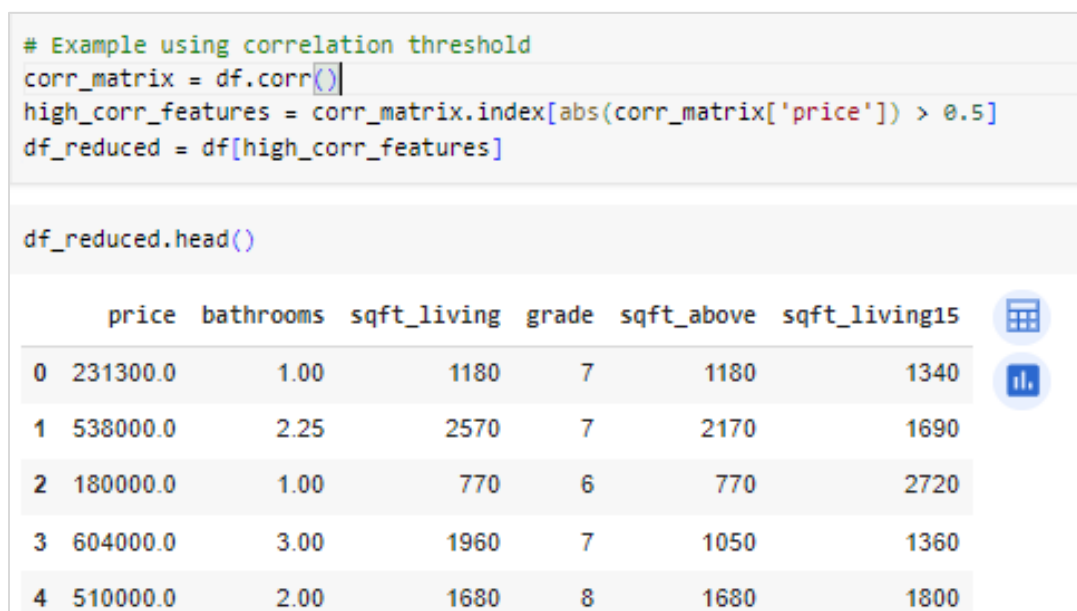


Figure 6 - Feature reduction based on correlation matrix.


```

# Initialize models
linear_model = LinearRegression()
random_forest_model = RandomForestRegressor(n_estimators=100, random_state=42)
knn_model = KNeighborsRegressor(n_neighbors=5)

# Separate features and target variable
X = df_reduced.drop('price', axis=1)
y = df_reduced['price']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the models
linear_model.fit(X_train, y_train)
random_forest_model.fit(X_train, y_train)
knn_model.fit(X_train, y_train)

```

Figure 7 - Creating and training the models.

```

# Predictions
linear_preds = linear_model.predict(X_test)
rf_preds = random_forest_model.predict(X_test)
knn_preds = knn_model.predict(X_test)

# Evaluation
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_preds))
linear_r2 = r2_score(y_test, linear_preds)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_preds))
rf_r2 = r2_score(y_test, rf_preds)

knn_rmse = np.sqrt(mean_squared_error(y_test, knn_preds))
knn_r2 = r2_score(y_test, knn_preds)

print(f'Linear Regression RMSE: {linear_rmse}, R^2: {linear_r2}')
print(f'Random Forest RMSE: {rf_rmse}, R^2: {rf_r2}')
print(f'KNN RMSE: {knn_rmse}, R^2: {knn_r2}')

```

Figure 8 - Predicting the models on test data.

```

Linear Regression RMSE: 261392.1693005368, R^2: 0.5480397400391172
Random Forest RMSE: 262715.474127974, R^2: 0.5434520349336687
KNN RMSE: 273653.16468823556, R^2: 0.5046455672573993

```

Figure 9 - RMSE, R2 values of the models.

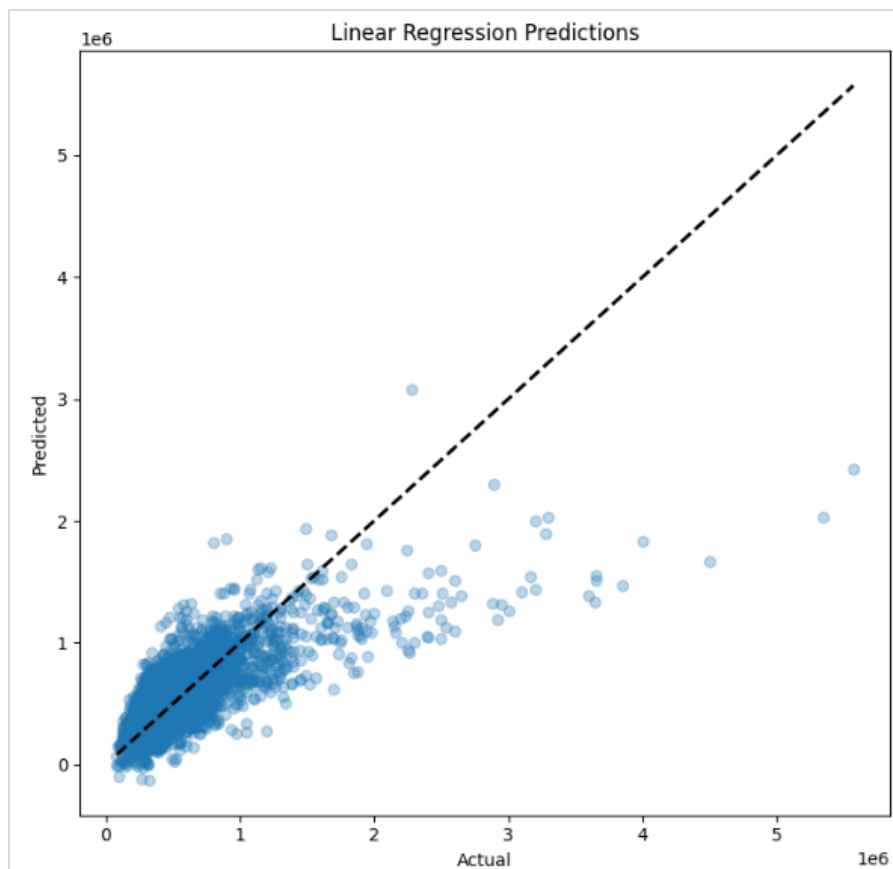


Figure 10 - Prediction Plot: Linear Regression

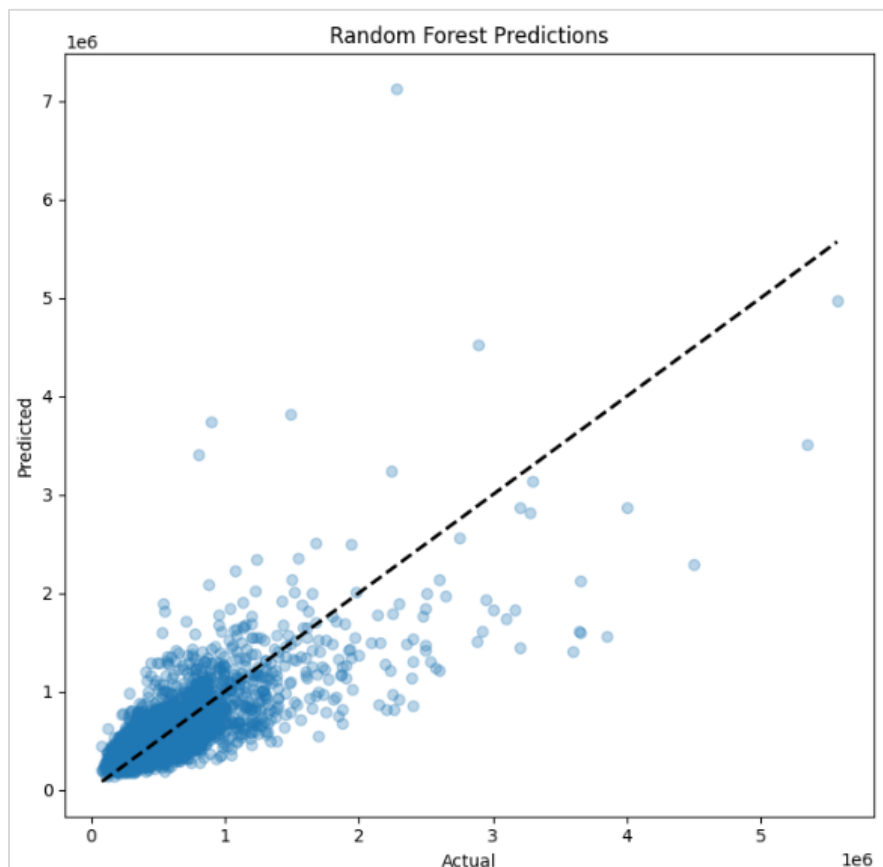


Figure 11 - Prediction Plot: Random Forests

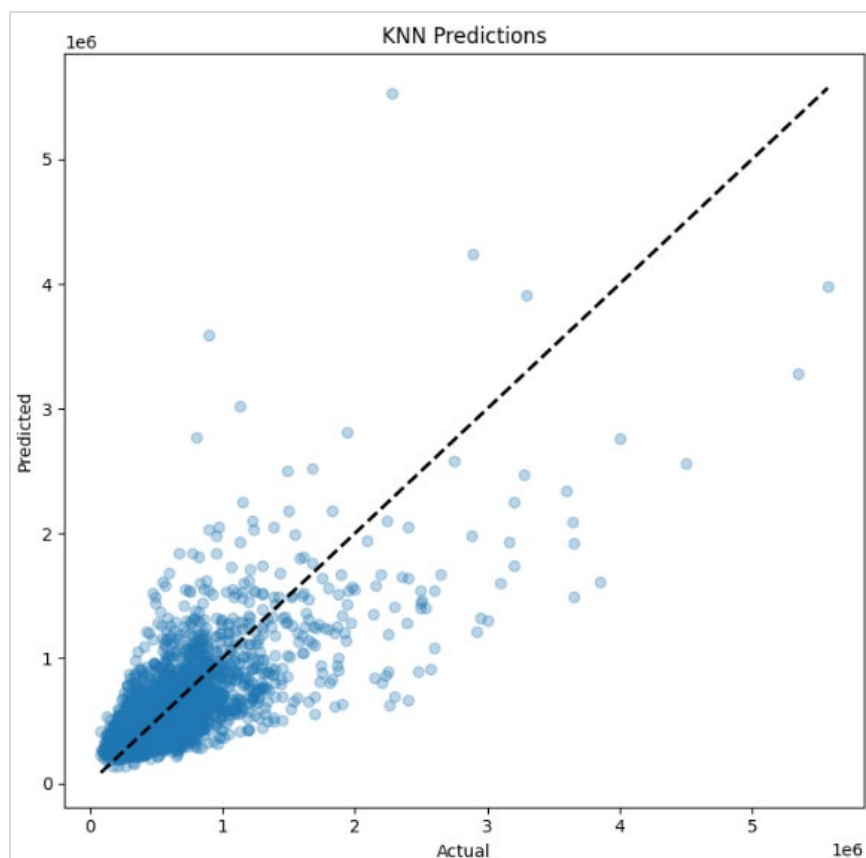


Figure 12 - Prediction Plot: KNN

Appendix B

Microsoft Azure AutoML Process Figures and Images

Default Directory > wkmlstudioAI510 > Training job

Submit a training job

1 Training method

2 Basic settings

3 Task type & data

4 Task settings

5 Compute

6 Review

How do you want to train your model?

Choose the method of training you would like to use.

 **Train automatically**

Submit an Automated ML job to train a model without writing a single line of code.

 **Run a custom training script**

Submit a command job to train your model using your own code.

 **Perform hyperparameter tuning**

Submit a sweep job that performs hyperparameter tuning using random, grid or bayesian sampling algorithms.

Back

Start configuring job

Figure 13 - Create training wizard dialog – step 2

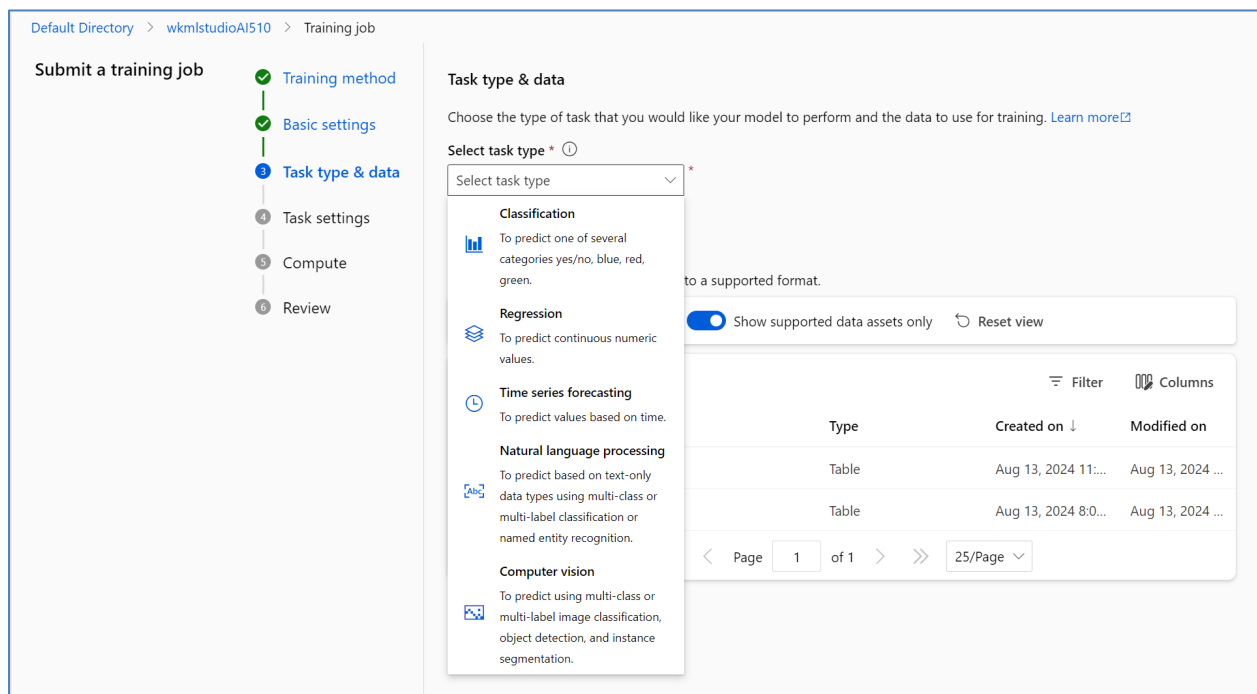


Figure 14 - Create training wizard dialog – step 3.

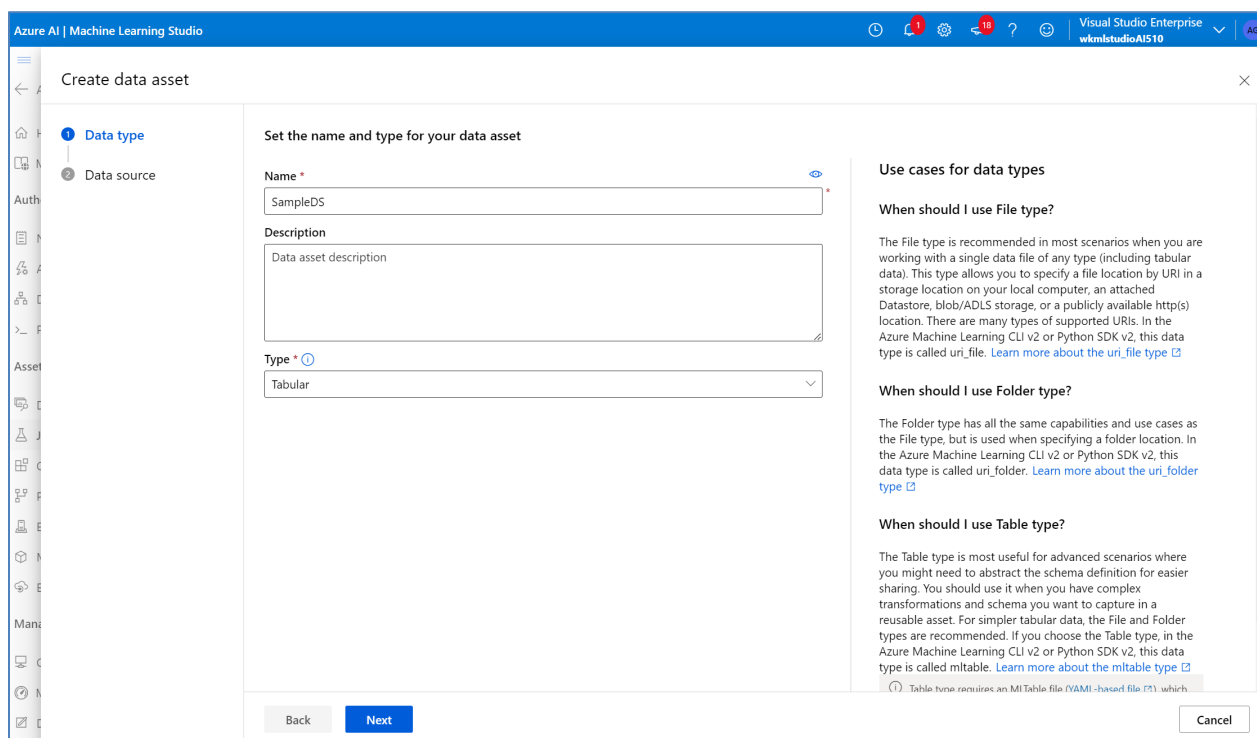


Figure 15 - Create dataset - data type.

Create data asset

✓ Data type

✓ Data source

✓ Destination storage type

✓ File or folder selection

⚙ Settings

📄 Schema

🔍 Review

Settings

These settings determine how the data is parsed. The initial settings are automatically detected; you can change them as needed to reparse the data.

File format

Delimited

Delimiter

Comma

Example

Field1,Field2,Field3

Encoding

UTF-8

Column headers

All files have same headers

Skip rows

None

☐ Dataset contains multi-line data

ⓘ Note: Processing tabular files with multi-line data is slower because multiple CPU cores cannot be used to ingest the data in parallel. Checking this option may result in slower processing times.

Data preview

bathro...	sqft_liv...	sqft_lot	waterfr...	view	condition	grade	sqft_ab...	sqft_ba...	yr_built	yr_reno...	zipcode	lat	long
3.75	4170	8142	0	2	3	10	4170	0	2006	0	98056	47.535	-122.181
2.75	2500	5995	0	0	3	8	2500	0	2008	0	98042	47.375	-122.107
2.25	1530	981	0	0	3	8	1480	50	2006	0	98103	47.653	-122.346
2.75	3600	9437	0	0	3	9	3600	0	2014	0	98059	47.482	-122.131
3.25	3410	10125	0	0	3	10	3410	0	2007	0	98040	47.565	-122.223

Back

Next

Review

Cancel

Figure 16 - Data selections, settings & schema.

Default Directory > wkmlstudioAI510 > Training job

Submit a training job

✓ Training method

✓ Basic settings

✓ Task type & data

⚙ Task settings

🔍 Compute

🔍 Review

Task settings

Task type

Regression

Data

housingpricevalidVer1 (View data)

Target column *

Select a target column

price (Decimal)

bathrooms (Decimal)

sqft_living (Integer)

sqft_lot (Integer)

waterfront (Integer)

view (Integer)

condition (Integer)

grade (Integer)

sqft_above (Integer)

sqft_basement (Integer)

yr_built (Integer)

yr_renovated (Integer)

zipcode (Integer)

lat (Decimal)

long (Decimal)

sqft_living15 (Integer)

sqft_lot15 (Integer)

Cancel

Figure 17 - Select target for predictions.

Page 20 of 38

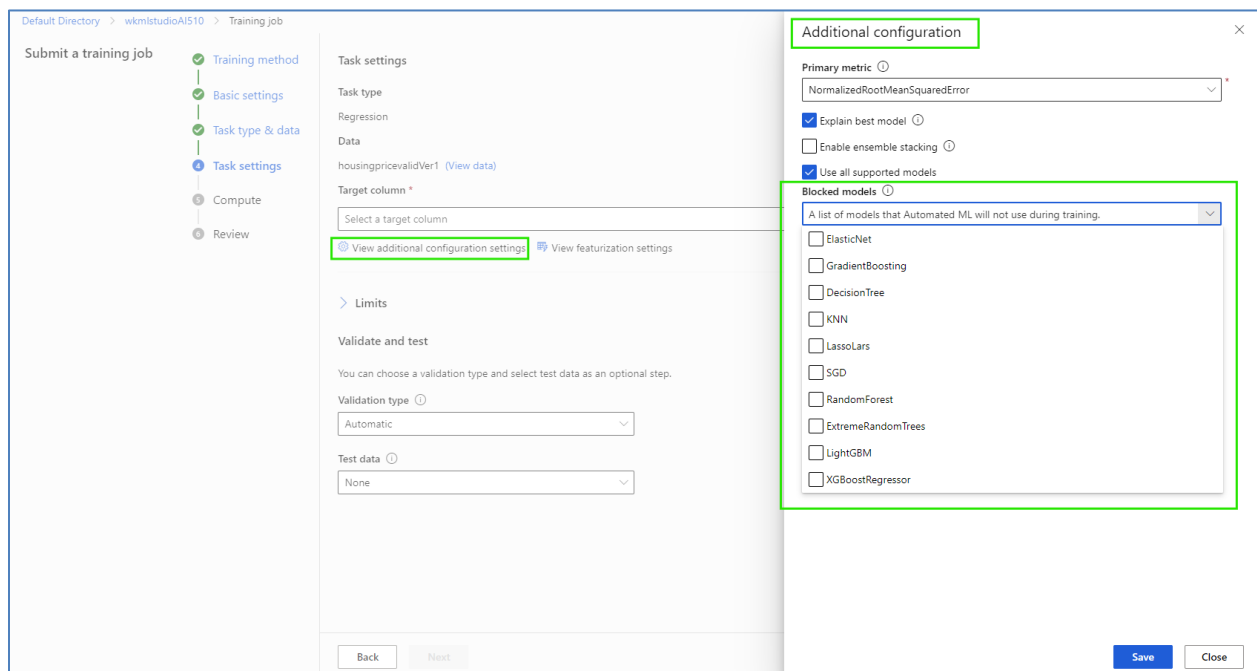


Figure 18 - Select additional settings.

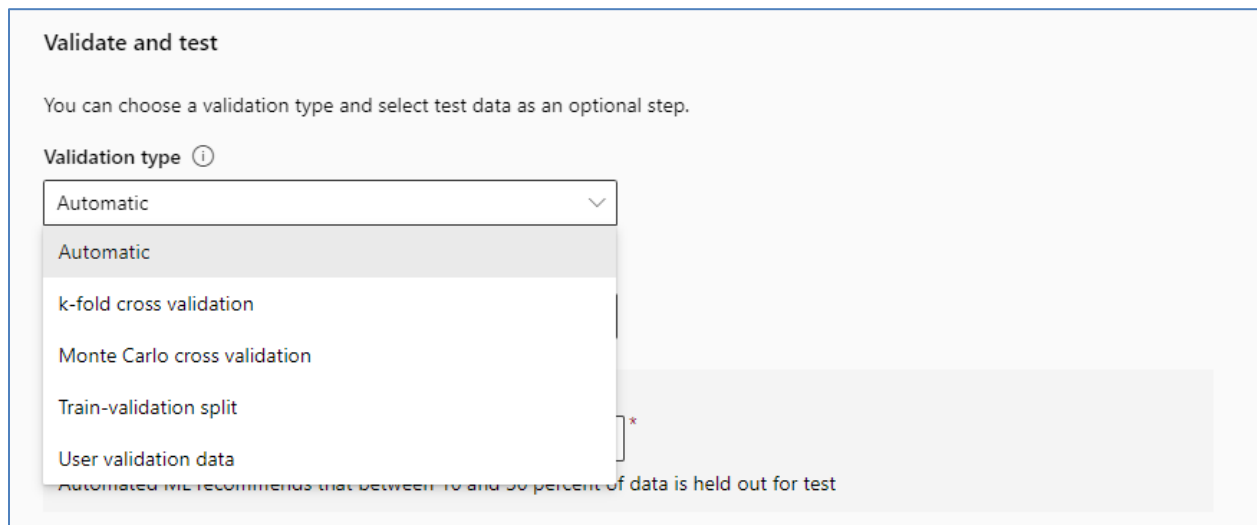


Figure 19 - Validation type selection.

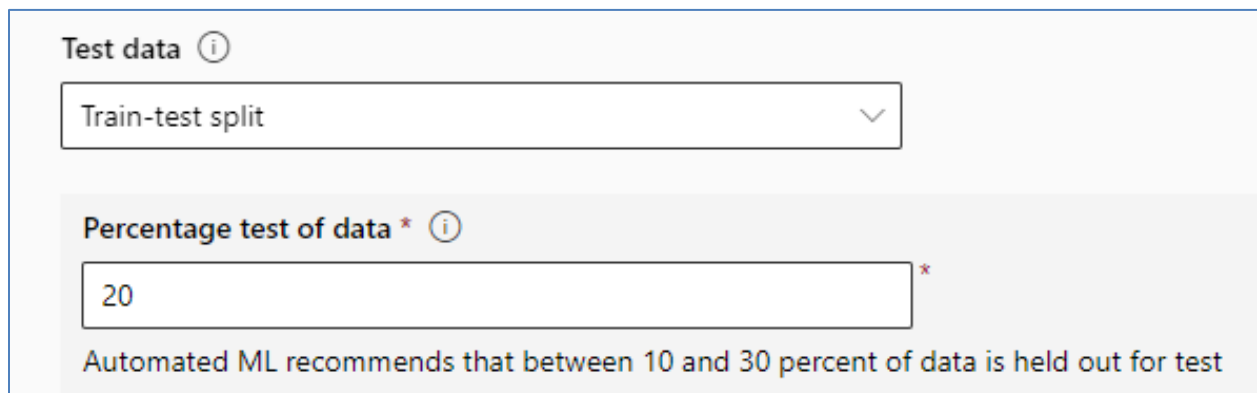


Figure 20 - Test-train split settings.

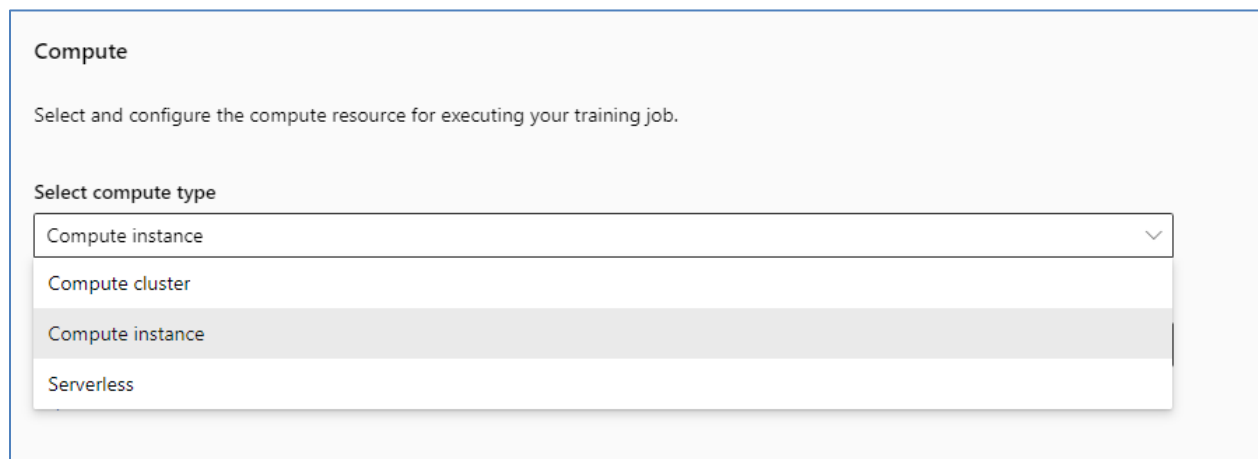


Figure 21 - Compute option selection.

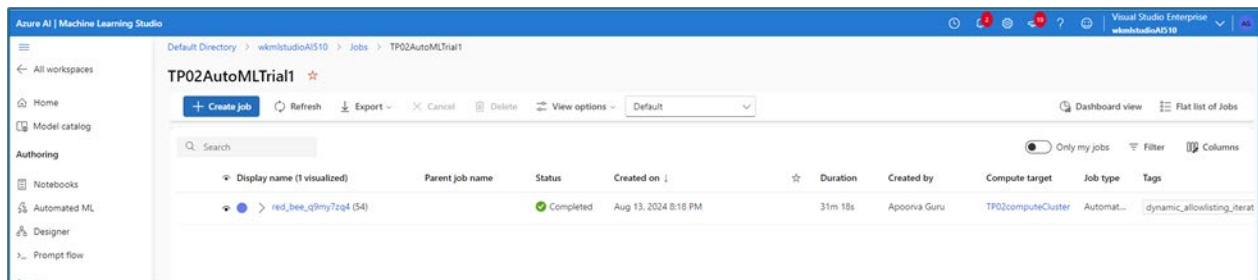


Figure 22 - Job submitted.

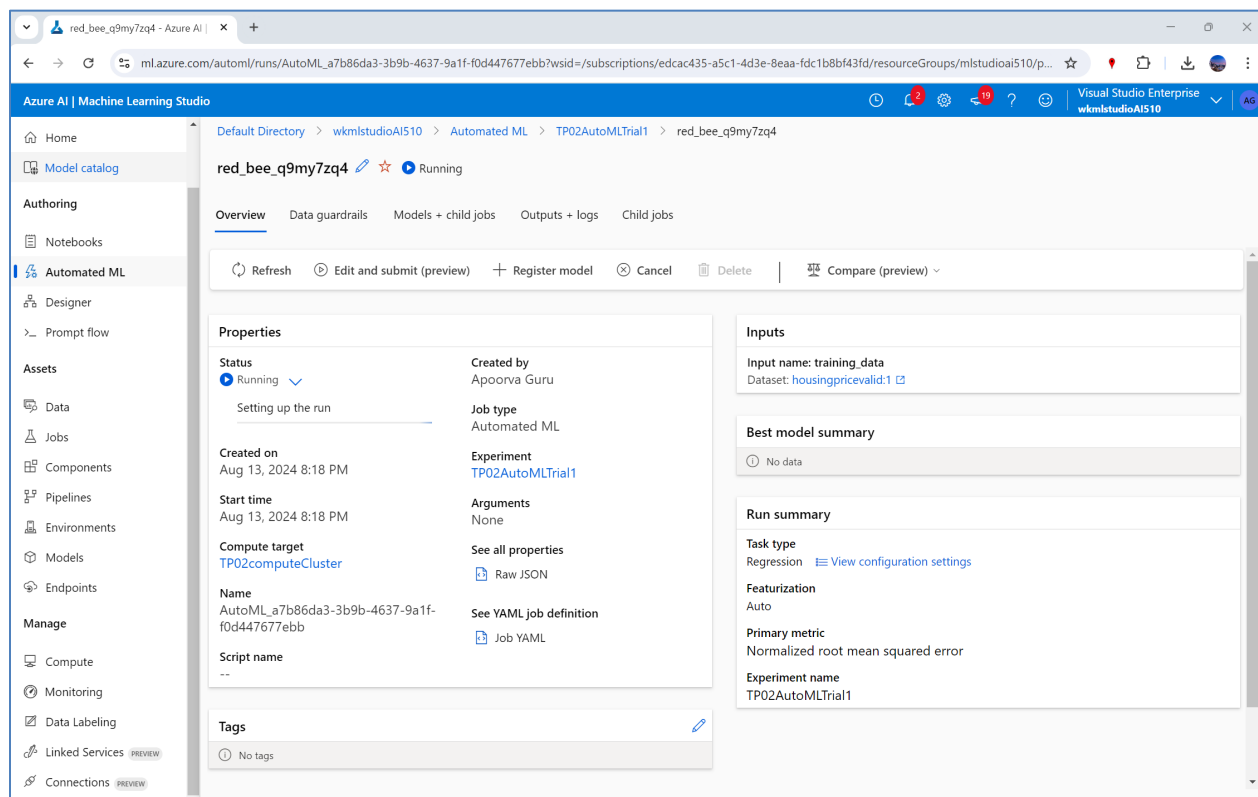


Figure 23 - Overview of running job.

red_bee_q9my7zq4 ✎ ★ ● Running

Overview Data guardrails **Models + child jobs** Outputs + logs Child jobs

Refresh Deploy Download Explain model # View generated code Reset view

Search Filter Columns

Algorithm name	Explained	Responsible AI	Normalized ro...	Sampling	Created on
MaxAbsScaler, XGBoostRegressor			0.01670	100.00 %	Aug 13, 2024 8:25 PM
MaxAbsScaler, LightGBM			0.01695	100.00 %	Aug 13, 2024 8:25 PM
StandardScalerWrapper, LightGBM			0.01703	100.00 %	Aug 13, 2024 8:35 PM
StandardScalerWrapper, XGBoostRegressor			0.01782	100.00 %	Aug 13, 2024 8:42 PM
MaxAbsScaler, LightGBM			0.01978	100.00 %	Aug 13, 2024 8:25 PM
MaxAbsScaler, LightGBM			0.02201	100.00 %	Aug 13, 2024 8:39 PM
MaxAbsScaler, ExtremeRandomTrees			0.02346	100.00 %	Aug 13, 2024 8:25 PM
MaxAbsScaler, ExtremeRandomTrees			0.02442	100.00 %	Aug 13, 2024 8:41 PM
StandardScalerWrapper, ElasticNet			0.02508	100.00 %	Aug 13, 2024 8:25 PM
StandardScalerWrapper, ElasticNet			0.02508	100.00 %	Aug 13, 2024 8:25 PM

Page 1 of 2 25/Page

Figure 24 - AutoML Model Choices - pg. 1 of 2

red_bee_q9my7zq4 ✎ ★ ● Running

Overview Data guardrails **Models + child jobs** Outputs + logs Child jobs

Refresh Deploy Download Explain model # View generated code Reset view

Search Filter Columns

Algorithm name	Explained	Responsible AI	Normalized ro...	Sampling	Created on
StandardScalerWrapper, ElasticNet			0.02762	100.00 %	Aug 13, 2024 8:25 PM
TruncatedSVDWrapper, ElasticNet			0.02763	100.00 %	Aug 13, 2024 8:35 PM
MaxAbsScaler, ElasticNet			0.02790	100.00 %	Aug 13, 2024 8:25 PM
StandardScalerWrapper, ElasticNet			0.02944	100.00 %	Aug 13, 2024 8:25 PM
StandardScalerWrapper, DecisionTree			0.03049	100.00 %	Aug 13, 2024 8:25 PM
StandardScalerWrapper, ElasticNet			0.03099	100.00 %	Aug 13, 2024 8:25 PM
MaxAbsScaler, GradientBoosting			0.03120	100.00 %	Aug 13, 2024 8:37 PM
MaxAbsScaler, ExtremeRandomTrees			0.03206	100.00 %	Aug 13, 2024 8:43 PM
MaxAbsScaler, ExtremeRandomTrees			0.03264	100.00 %	Aug 13, 2024 8:44 PM
MaxAbsScaler, ExtremeRandomTrees			0.03312	100.00 %	Aug 13, 2024 8:44 PM

Page 2 of 2 25/Page

Figure 25 - AutoML Model Choices - pg. 2 of 2

Default Directory > wmlstudioAI510 > Jobs > TP02AutoMLTrial1 > red_bee_q9my7zq4

red_bee_q9my7zq4 Completed

Overview Data guardrails **Models + child jobs** Outputs + logs Child jobs

Refresh Deploy Download Explain model # View generated code Reset view

Search Filter Columns

Algorithm name	Explained	Responsible AI	Normalized root mean squared error ↑	Sampling	Created on	Duration	Hyperparameter
VotingEnsemble	View explanation		0.01584	100.00 %	Aug 13, 2024 8:48 PM	50s	algorithm : [XG
StackEnsemble			0.01591	100.00 %	Aug 13, 2024 8:48 PM	52s	algorithm : [XG
MaxAbsScaler, XGBoostRegressor			0.01670	100.00 %	Aug 13, 2024 8:25 PM	30s	tree_method : a
MaxAbsScaler, LightGBM			0.01695	100.00 %	Aug 13, 2024 8:25 PM	28s	min_data_in_lea
StandardScalerWrapper, LightGBM			0.01703	100.00 %	Aug 13, 2024 8:35 PM	41s	boosting_type :
StandardScalerWrapper, XGBoostRegressor			0.01782	100.00 %	Aug 13, 2024 8:42 PM	43s	booster : gbtree
MaxAbsScaler, LightGBM			0.01978	100.00 %	Aug 13, 2024 8:25 PM	24s	boosting_type :
MaxAbsScaler, LightGBM			0.02201	100.00 %	Aug 13, 2024 8:39 PM	39s	boosting_type :
MaxAbsScaler, ExtremeRandomTrees			0.02346	100.00 %	Aug 13, 2024 8:25 PM	28s	bootstrap crit
MaxAbsScaler, RandomForest			0.02374	100.00 %	Aug 13, 2024 8:45 PM	1m 1s	bootstrap : true
MaxAbsScaler, ExtremeRandomTrees			0.02442	100.00 %	Aug 13, 2024 8:41 PM	43s	bootstrap crit

Figure 26 - Best model based on normalized Root Mean Squared Error value.

Azure AI | Machine Learning Studio

funny_station_v7gsI94f Completed

Overview **Model** Explanations (preview) Responsible AI (preview) Metrics Data transformation (preview) Test results (preview) Outputs

Refresh Deploy Download Explain model # View generated code Test model (preview) Register model

Model summary

Algorithm name
VotingEnsemble

Ensemble details
[View ensemble details](#)

Normalized root mean squared error
0.01494 [View all other metrics](#)

Sampling
100.00 %

Registered models
tp02automltrial285:1

Deploy status
housingpricepredservice Succeeded
housingpricepred Succeeded

Run Metrics

- Explained variance
0.90053
- Mean absolute error
64229
- Mean absolute percentage error
12.354
- Median absolute error
38086
- Normalized mean absolute error
0.0084235
- Normalized median absolute error
0.0049949**
- Normalized root mean squared error
0.014941
- Normalized root mean squared log error
0.035939
- R2 score
0.90052
- Root mean squared error
1.1393e+5
- Root mean squared log error
0.16645
- Spearman correlation
0.94997

Figure 27 - Additional metrics.

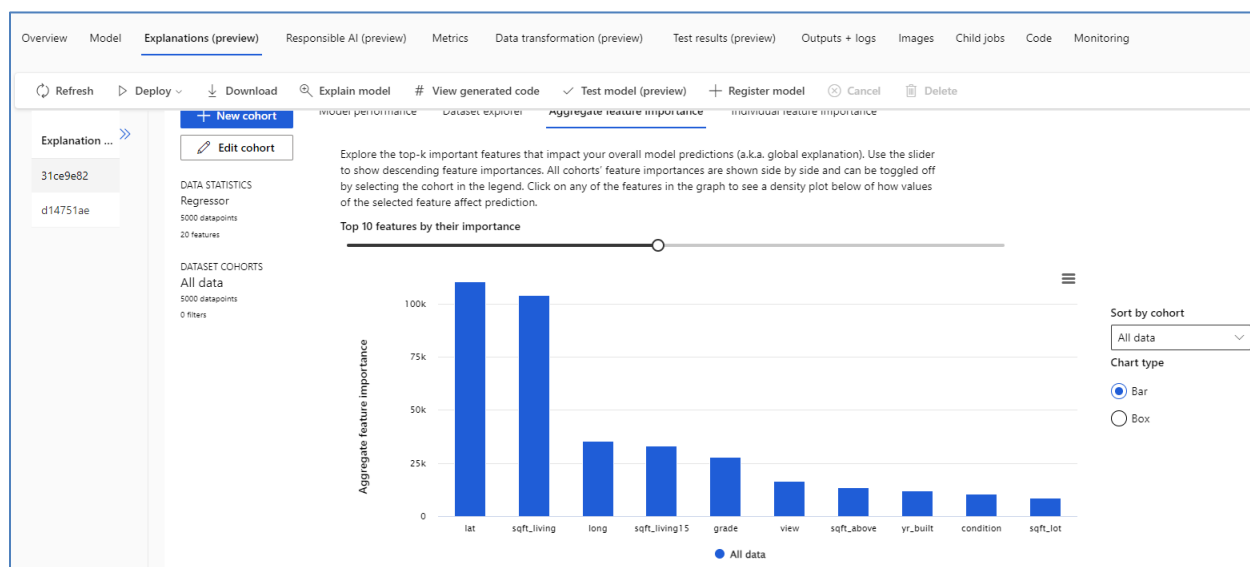


Figure 28 - Top 10 features sorted by relevance (highest to lowest)

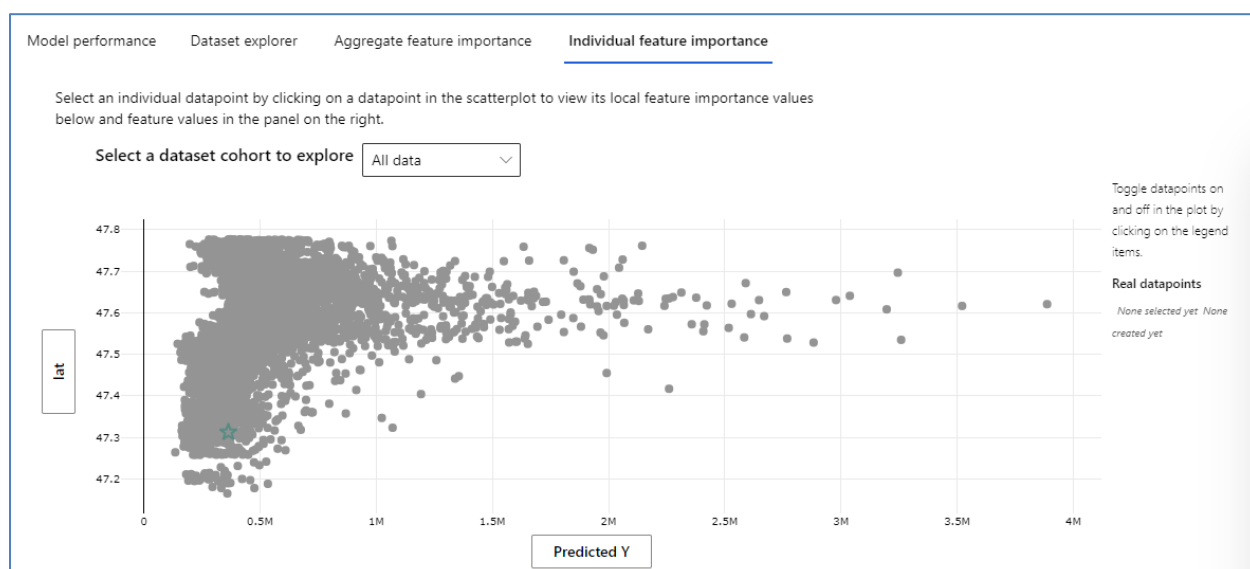


Figure 29 - Latitude vs Price: Most significant feature.

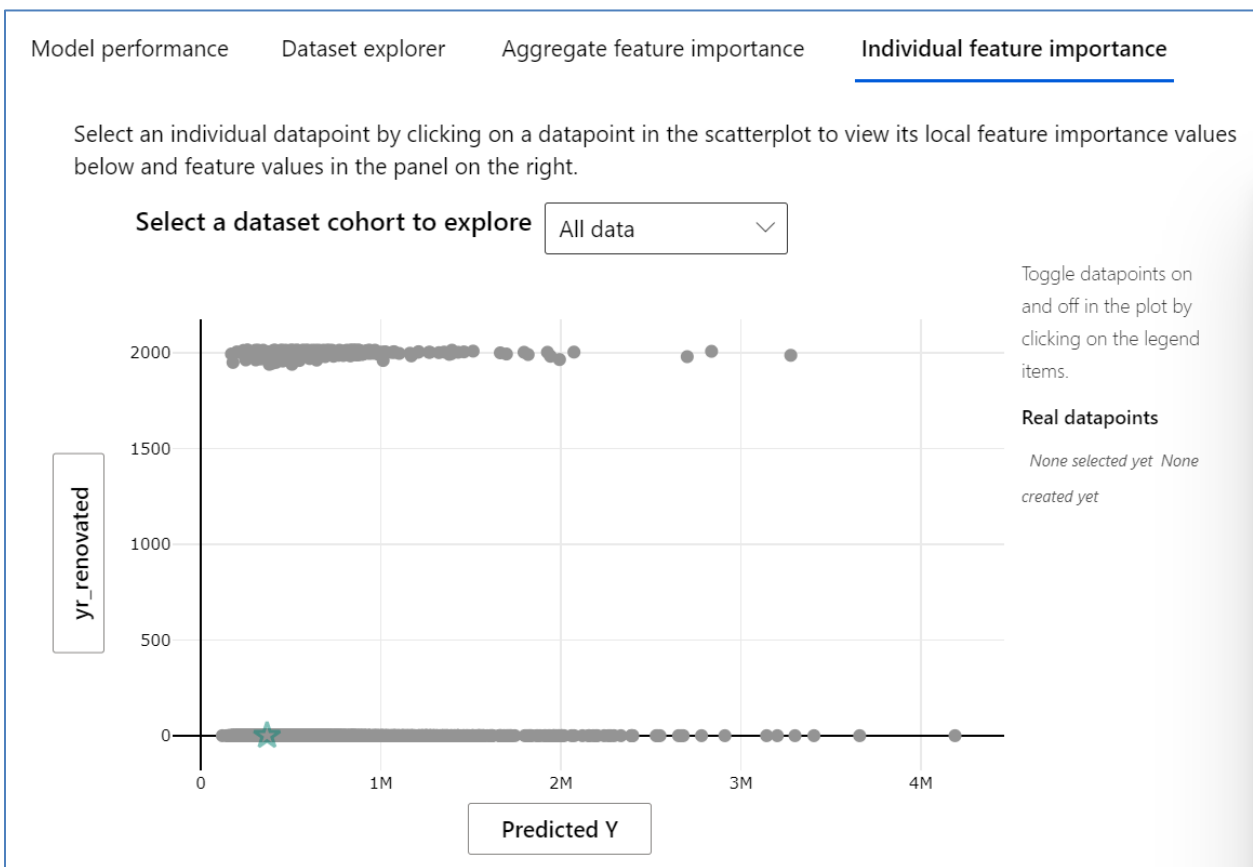


Figure 30 - Year-Renovated vs Price: Least significant feature.

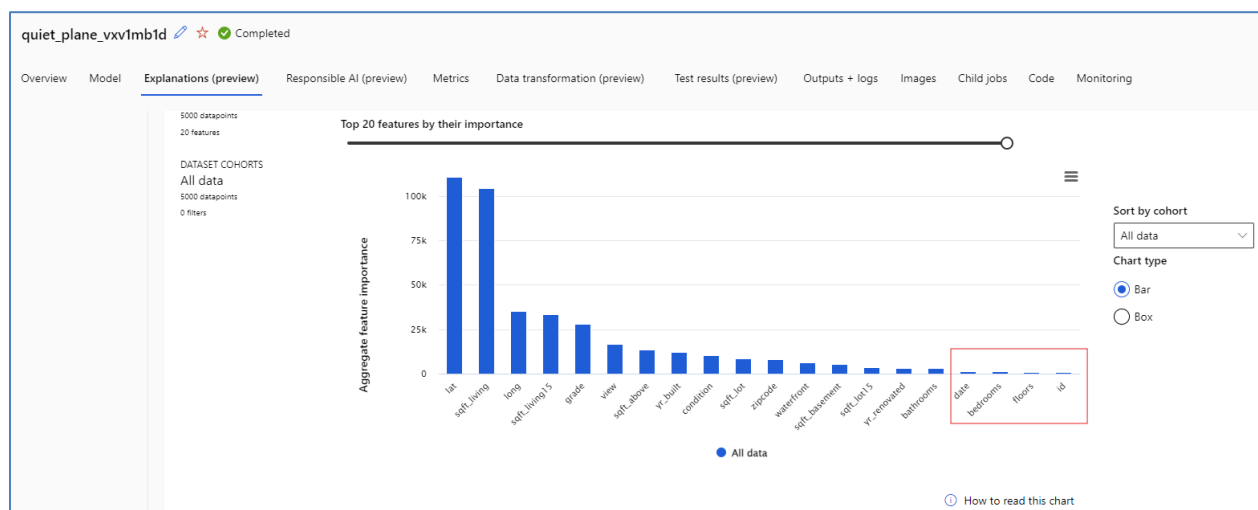


Figure 31 - Features removed based on top 20 features by importance.

Deploy a model

Name * ⓘ

Description ⓘ

Compute type * ⓘ

AksCompute

Compute name * ⓘ

Select or search by name

Models: tp02automltrial285:1

Enable authentication

☒

Type

Token-based authentication

Token-based authentication

Key-based authentication

Use custom deployment assets

☐ Use custom deployment assets

> Advanced

Deploy

Cancel

Figure 32 - Model deployment & access security settings.

Algorithm name	Explained	Responsible AI	Normalized root mean squared error ↑	Sampling	Created on	Duration	Hyperparameter
VotingEnsemble	View explanation		0.01494	100.00 %	Aug 14, 2024 6:02 AM	1m 23s	algorithm : [XG
StandardScalerWrapper, XGBoostRegressor			0.01629	100.00 %	Aug 14, 2024 4:39 AM	50s	boosters : gbtree
StandardScalerWrapper, XGBoostRegressor			0.01629	100.00 %	Aug 14, 2024 3:53 AM	50s	boosters : gbtree
MaxAbsScaler, LightGBM			0.01643	100.00 %	Aug 14, 2024 2:47 AM	47s	boosting_type :

Figure 33 - Successful and secure deployment of model.

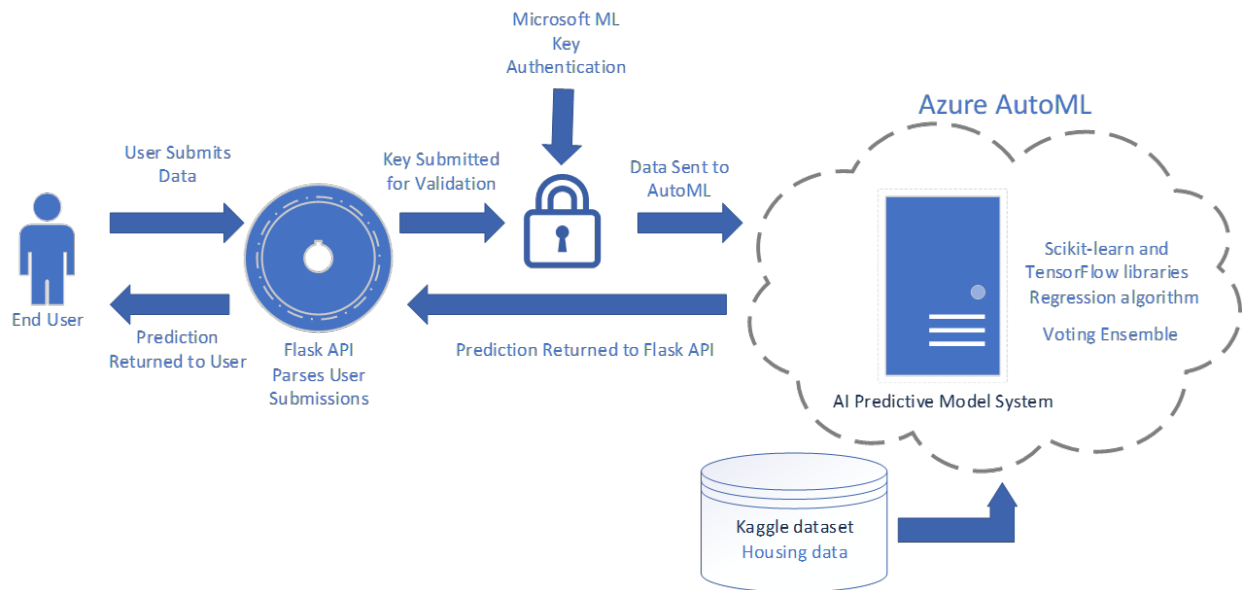


Figure 34 - Key-Based Authentication process.

Endpoint attributes	Tags	Properties
Service ID housingpricepredservice Description -- Deployment state Healthy Operation state Succeeded Compute type Container instance Created by Apoorva Guru Model ID tp02automltrial285.1 Created on Aug 21, 2024 9:27 PM Last updated on Aug 21, 2024 9:27 PM Image ID --	No data	runid tp02automltrial2_285 hasInferenceSchema True hashttps False authEnabled True

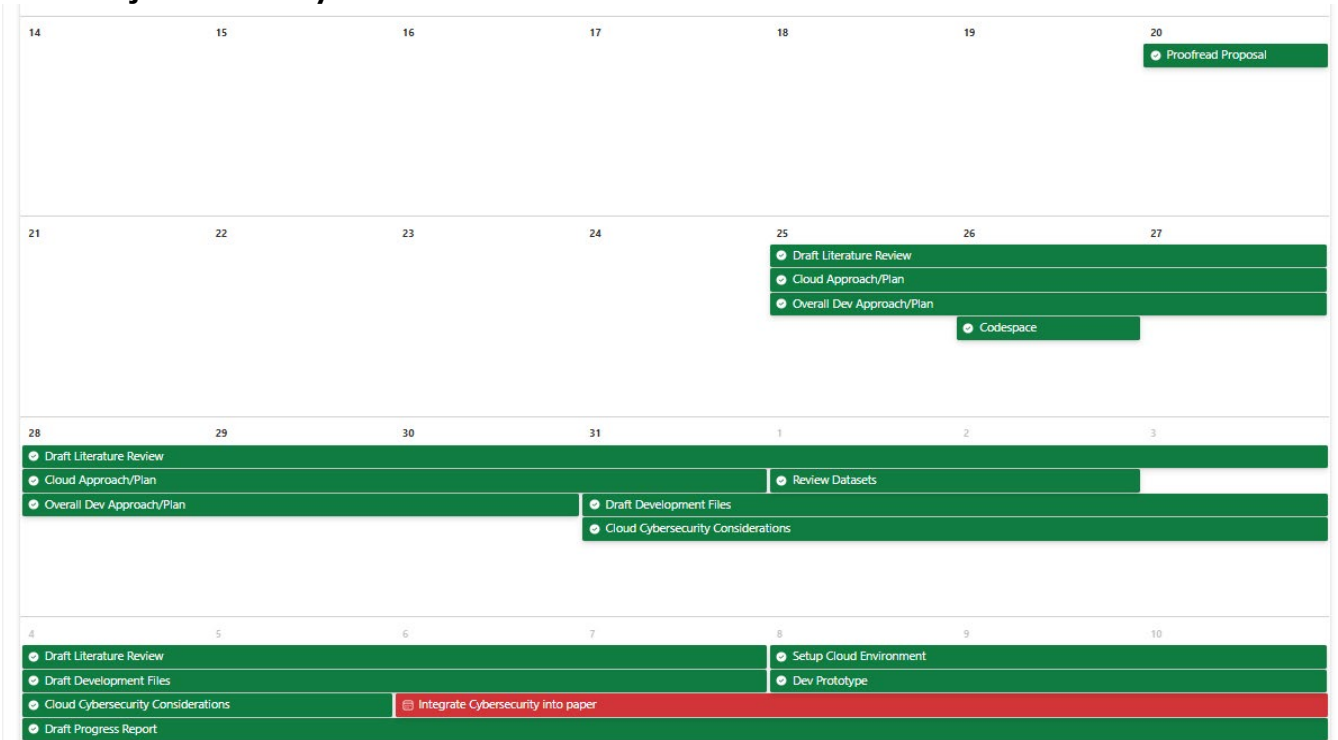
REST endpoint
<http://3a271271-181e-422a-9847-52137b72e7c5.westus3.azurecontainer.io/score>

Figure 35 - AutoML web service details including REST endpoint.

Appendix C Team Project Plan

Microsoft Planner is tracking the team's plan and progress throughout this team project.

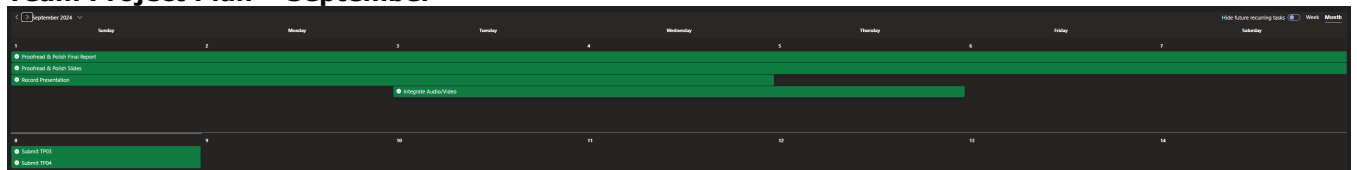
Team Project Plan - July



Team Project Plan – August



Team Project Plan – September



Team Project Progress – Kanban buckets

AI Cloud Computing Project Plan

GridBoardScheduleCharts

TP 03 Final Report

+ Add task

Completed tasks10

✓ Refine Dev/Cloud Product

Completed by Stanley George on...

✓ Integrate Front-End & AutoML

Completed by Veronica Elze on 0...

✓ Run Metrics for Visualizations

Run Metrics until satisfactory or at least con

Capture metrics in image format for present

0 / 2

Completed by Stanley George on...

✓ Draft Final Report

4 / 4

Completed by Teagan Scharlau o...

✓ Proofread & Polish Final Report

Apoorva's review complete?

3 / 4

Completed by Veronica Elze on 0...

✓ Setup Cloud Environment

Completed by Teagan Scharlau o...

✓ Submit TP03

!

Completed by Veronica Elze on 0...

✓ Test Dev Prototype

Completed by Veronica Elze on 0...

✓ Test Integrated Cloud Setup

Completed by Teagan Scharlau o...

© Open Enterprise from Source for the individual project and its associated code

✓ Draft Metrics

Metrics.docx

TP 04 Presentation

+ Add task

Completed tasks5

✓ Draft Presentation Slides

Completed by Veronica Elze on 0...

✓ Proofread & Polish Slides

Apoorva's review complete?

3 / 4

Completed by Veronica Elze on 0...

✓ Record Presentation

4 / 4

Completed by Veronica Elze on 0...

✓ Integrate Audio/Video

Completed by Veronica Elze on 0...

✓ Submit TP04

!

Completed by Veronica Elze on 0...

DONE - TP 01 Project Proposal

+ Add task

Completed tasks9

✓ Proofread Proposal

Completed by Veronica Elze on 0...

✓ Proposal Template

Completed by Veronica Elze on 0...

✓ Scholarly References

Completed by Veronica Elze on 0...

✓ Workload Assignments

Completed by Veronica Elze on 0...

✓ Project Tool(s)

Completed by Veronica Elze on 0...

✓ Usefulness

Completed by Veronica Elze on 0...

✓ Project Summary

Completed by Veronica Elze on 0...

✓ Project Description

Completed by Veronica Elze on 0...

✓ Team Charter

Completed by Veronica Elze on 0...

DONE - TP 02 Progress Report

+ Add task

Completed tasks15

✓ Draft Front-End

Completed by Veronica Elze on 0...

✓ Integrate Planner into Appendix

Completed by Veronica Elze on 0...

✓ Cloud Approach/Plan

3 / 3

Completed by Teagan Scharlau o...

✓ Codespace

Each team member sends Verónica usernam

2 / 3

Completed by Veronica Elze on 0...

✓ Draft Literature Review

We each identify at least one scholarly sourc

12 / 3

Completed by Veronica Elze on 0...

✓ Review Datasets

2 / 2

Completed by Teagan Scharlau o...

✓ Evaluate AutoML

Completed by Teagan Scharlau o...

✓ Cloud Cybersecurity Considerations

111

Completed by Stanley George on...

✓ Overall Dev Approach/Plan

3 / 3

Completed by Teagan Scharlau o...

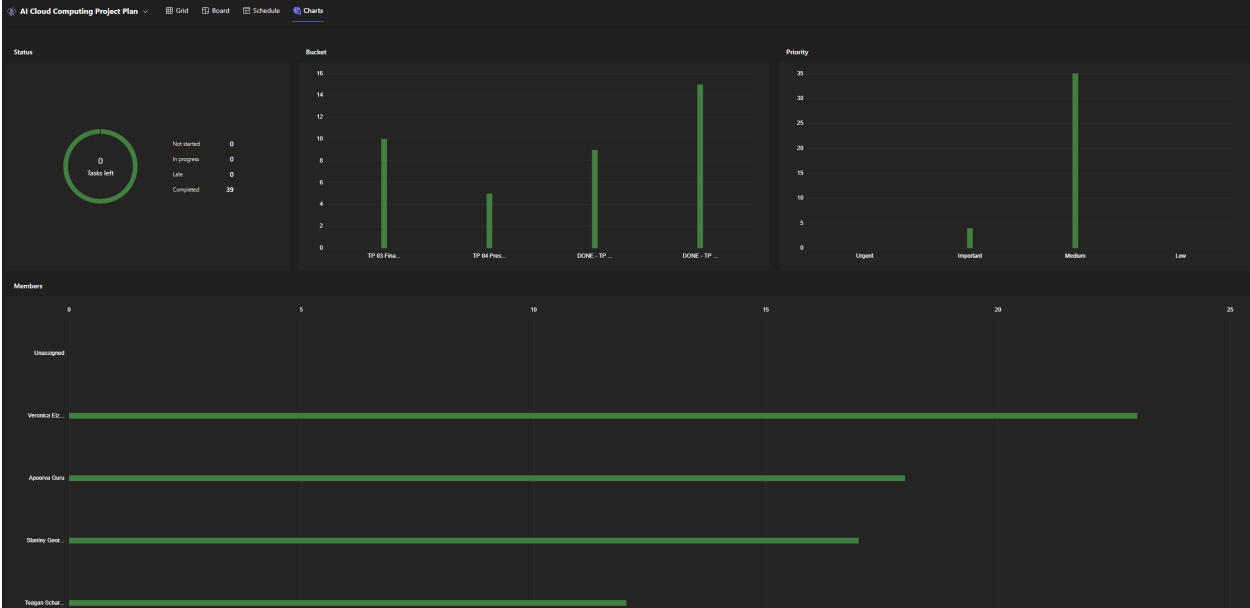
✓ Dev Prototype

Completed by Veronica Elze on 0...

✓ Draft Development Files

Completed by Veronica Elze on 0...

Team Project Progress – Charts (quantity, bucket division, priorities, and assignment allocations)



Filename: index.html

Function: Collect user input, display predicted output

Author: Veronica Elze

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Predict House Price</title>
  <style>
    /* Basic styling for the body */
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
      margin: 0;
      padding: 0;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    /* Styling for the container div */
    .container {
      background-color: #fff;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
      width: 300px;
    }
    /* Styling for the heading */
    h1 {
      text-align: center;
      color: #333;
    }
    /* Styling for the labels */
    label {
      display: block;
      margin-bottom: 5px;
      color: #555;
    }
    /* Styling for the text inputs */
    input[type="text"] {
      width: 95%;
      padding: 8px;
      margin-bottom: 10px;
      border: 1px solid #ccc;
      border-radius: 4px;
    }
    /* Styling for the submit button */
    input[type="submit"] {
      width: 100%;
      padding: 10px;
      background-color: #28a745;
      border: none;
      border-radius: 4px;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>House Price Predictor</h1>
    <label>Enter house size (sq ft):</label>
    <input type="text">
    <label>Enter number of bedrooms:</label>
    <input type="text">
    <label>Enter number of bathrooms:</label>
    <input type="text">
    <input type="submit" value="Predict Price">
  </div>
</body>
</html>
```

```

        color: #fff;
        font-size: 16px;
        cursor: pointer;
    }
    /* Hover effect for the submit button */
    input[type="submit"]:hover {
        background-color: #218838;
    }
    /* Responsive styling for smaller screens */
    @media (max-width: 600px) {
        .container {
            width: 90%;
            padding: 15px;
        }
        input[type="text"], input[type="submit"] {
            font-size: 14px;
        }
    }
}
</style>
</head>
<body>
    <div class="container">
        <h1>Predict House Price</h1>
        <form action="/predict" method="post">
            {% for feature in features %}
            <!-- Loop through features and create a label and input for each -->
            <label for="{{ feature }}">{{ feature_labels[feature] }}</label>
            <input type="text" id="{{ feature }}" name="{{ feature }}">
            {% endfor %}
            <!-- Submit button for the form -->
            <input type="submit" value="Predict">
        </form>
    </div>
</body>
</html>

```

Filename: app.py (manual)

Function: Implements Flask API, connects UI to localhost, formats data

Author: Verónica Elze

```

from flask import Flask, request, render_template
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

```

```

app = Flask(__name__)

```

```

# Create feature labels
feature_labels = {
    'bedrooms': 'Number of Bedrooms',
    'bathrooms': 'Number of Bathrooms',
    'sqft_living': 'Living Area (sqft)',
    'sqft_lot': 'Lot Area (sqft)',
    'floors': 'Number of Floors',
    'waterfront': 'Waterfront (Yes=1, No=0)',
    'view': 'View Rating',
    'condition': 'Condition Rating',
}

```

```

'grade': 'Grade Rating',
'sqft_above': 'Above Ground Living Area (sqft)',
'sqft_basement': 'Basement Area (sqft)',
'yr_built': 'Year Built',
'yr_renovated': 'Year Renovated',
'zipcode': 'Zip Code',
'lat': 'Latitude',
'long': 'Longitude',
'sqft_living15': 'Living Area of 15 Nearest Neighbors (sqft)',
'sqft_lot15': 'Lot Area of 15 Nearest Neighbors (sqft)'
}

# Load the dataset and model
url = "housing.csv"
df = pd.read_csv(url)
df = df.drop(['id', 'date'], axis=1)

# Replace empty strings with NaN and then drop or fill them
df.replace("", np.nan, inplace=True)
df.dropna(inplace=True)

# Calculate the correlation matrix
corr_matrix = df.corr()

# Print the correlation matrix for debugging
print(corr_matrix)

# Adjust the threshold to find highly correlated features
threshold = 0.5
high_corr_features = corr_matrix.index[abs(corr_matrix["price"]) > threshold].tolist()

# Check if 'price' is in the list before removing it
if 'price' in high_corr_features:
    high_corr_features.remove('price')

filtered_feature_labels = {feature: label for feature, label in feature_labels.items() if feature
in high_corr_features}

# Check if high_corr_features is not empty
if not high_corr_features:
    raise ValueError(f"No features found with correlation to 'price' above {threshold}.")

# Prepare the data
X = df[high_corr_features]
y = df['price']

# Check for missing values
if X.isnull().values.any() or y.isnull().values.any():
    raise ValueError("Missing values found in the dataset.")

# Train the model
model = LinearRegression()
model.fit(X, y)

# Dynamically pass feature labels
@app.route('/')
def index():

```

```

    return render_template('index.html', features=high_corr_features,
                           feature_labels=filtered_feature_labels)

# Prediction logic
@app.route('/predict', methods=['POST'])
def predict():
    input_data = [float(request.form[feature]) for feature in high_corr_features]
    input_data = np.array(input_data).reshape(1, -1)
    prediction = model.predict(input_data)[0]
    return f'The predicted house price is: ${prediction:.2f}'

if __name__ == '__main__':
    app.run(debug=True)

```

Filename: trainevalmodels.py

Function: Loads, trains, and evaluates models

Author: Apoorva Guru & Verónica Elze

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

# Import necessary machine learning models and metrics
from sklearn.ensemble import RandomForestRegressor # Add this import
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.neighbors import KNeighborsRegressor

# Load the dataset
url = "Housing.csv"
df = pd.read_csv(url)

# Drop unnecessary columns
df = df.drop(['id', 'date'], axis=1)

# Plot the distribution of housing prices
sns.histplot(df['price'], bins=50, kde=True)
plt.title('Distribution of Housing Prices')
plt.show()

# Plot the correlation matrix
plt.figure(figsize=(14, 10))
corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix', size=20)
plt.show()

# Select features with high correlation to price
corr_matrix = df.corr()
high_corr_features = corr_matrix.index[abs(corr_matrix['price']) > 0.5]
df_reduced = df[high_corr_features]

# Initialize models
linear_model = LinearRegression()
random_forest_model = RandomForestRegressor(n_estimators=100, random_state=42)

```



```

knn_model = KNeighborsRegressor(n_neighbors=5)

# Define features (X) and target (y)
X = df_reduced.drop('price', axis=1)
y = df_reduced['price']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the models
linear_model.fit(X_train, y_train)
random_forest_model.fit(X_train, y_train)
knn_model.fit(X_train, y_train)

# Make predictions
linear_preds = linear_model.predict(X_test)
rf_preds = random_forest_model.predict(X_test)
knn_preds = knn_model.predict(X_test)

# Calculate performance metrics
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_preds))
linear_r2 = r2_score(y_test, linear_preds)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_preds))
rf_r2 = r2_score(y_test, rf_preds)

knn_rmse = np.sqrt(mean_squared_error(y_test, knn_preds))
knn_r2 = r2_score(y_test, knn_preds)

# Print performance metrics
print(f'Linear Regression RMSE: {linear_rmse}, R^2: {linear_r2}')
print(f'Random Forest RMSE: {rf_rmse}, R^2: {rf_r2}')
print(f'KNN RMSE: {knn_rmse}, R^2: {knn_r2}')

# Plot predictions vs actual values
plt.figure(figsize=(21, 7))

plt.subplot(1, 3, 1)
plt.scatter(y_test, linear_preds, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Linear Regression Predictions')

plt.subplot(1, 3, 2)
plt.scatter(y_test, rf_preds, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Random Forest Predictions')

plt.subplot(1, 3, 3)
plt.scatter(y_test, knn_preds, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('KNN Predictions')

```

```
plt.tight_layout()
plt.show()
```

Filename: app.py

Function: Implements Flask API, connects UI to AutoML, formats data

Author: Verónica Elze

```
from flask import Flask, request, render_template, jsonify
import urllib.request
import json
import os
import ssl
import locale

# Create a Flask application instance
app = Flask(__name__)

# Function to allow self-signed HTTPS certificates
def allowSelfSignedHttps(allowed):
    if allowed and not os.environ.get('PYTHONHTTPSVERIFY', '') and getattr(ssl,
'_create_unverified_context', None):
        ssl._create_default_https_context = ssl._create_unverified_context

# Allow self-signed HTTPS certificates
allowSelfSignedHttps(True)

# Set locale for currency formatting
locale.setlocale(locale.LC_ALL, 'en_US.UTF-8')

# Define the home route
@app.route('/')
def home():
    return render_template('index.html') # Render the home page

# Define the predict route to handle POST requests
@app.route('/predict', methods=['POST'])
def predict():
    # Collect data from the form and prepare it for the API request
    data = {
        "Inputs": {
            "data": [
                {
                    "bathrooms": float(request.form['bathrooms']),
                    "sqft_living": float(request.form['sqft_living']),
                    "sqft_lot": float(request.form['sqft_lot']),
                    "waterfront": int(request.form['waterfront']),
                    "view": int(request.form['view']),
                    "condition": int(request.form['condition']),
                    "grade": int(request.form['grade']),
                    "sqft_above": float(request.form['sqft_above']),
                    "sqft_basement": float(request.form['sqft_basement']),
                    "yr_built": int(request.form['yr_built']),
                    "yr_renovated": int(request.form['yr_renovated']),
                    "zipcode": int(request.form['zipcode']),
                    "lat": float(request.form['lat']),
```

```

        "long": float(request.form['long']),
        "sqft_living15": float(request.form['sqft_living15']),
        "sqft_lot15": float(request.form['sqft_lot15'])
    }
]
}
}

# Convert the data to a JSON string
body = str.encode(json.dumps(data))
# URL of the API endpoint
url = 'http://3a271271-181e-422a-9847-52137b72e7c5.westus3.azurecontainer.io/score'
# API key for authentication
api_key = 'J0QTCd3P4ePk0bf5xV0FAbqLeODihzC5'
# Headers for the API request
headers = {'Content-Type': 'application/json', 'Authorization': ('Bearer ' + api_key)}

# Create a request object
req = urllib.request.Request(url, body, headers)

try:
    # Send the request and get the response
    response = urllib.request.urlopen(req)
    # Parse the JSON response
    result = json.loads(response.read())
    # Extract the prediction value
    prediction_value = result["Results"][0]
    # Format the prediction value as currency
    formatted_result = locale.currency(prediction_value, grouping=True)
    # Render the result page with the formatted prediction
    return render_template('result.html', result=formatted_result)
except urllib.error.HTTPError as error:
    # Handle HTTP errors
    return jsonify({"error": str(error)})

# Run the Flask application
if __name__ == '__main__':
    app.run(debug=True)

```